Rewriting the Universe

High-Performance Hypergraph Rewriting for the Wolfram Physics Project

Richard Assar
Wolfram Institute
rassar@wolframinstitute.org

October 31, 2026

Abstract

The Wolfram Physics Project models fundamental physics by hypergraph rewriting. Multiway evolution applies every applicable rewrite to every reachable state and records the states up to isomorphism, the events between them, and the causal and branchial relations over the events. For rules that add edges, the number of states grows super-exponentially with depth. We present a multiway hypergraph rewriting engine for CPU and GPU. Every state is identified by the hash of an exact McKay-style individualization–refinement canonical form, computed by one implementation shared by the CPU and the GPU's resident evolution kernel. The engine is lock-free, runs without phases or passes, maintains the causal and branchial relations and the causal graph's transitive reduction as events are created, and returns the same output at every thread count. Quotient exploration expands one state per isomorphism class and reconstructs the raw events and relations exactly. The concurrent data structures and protocols are model-checked under the RC11 memory model and its GPU-scoped extension, and four protocols are also model-checked under TLA+ for any number of workers. Against the `Wolfram/Multicomputation` paclet's `MultiwaySystem`, with identical state and causal-edge counts at every depth, the engine's C++ core is 1,696× faster at depth 4 and 50,384× faster at depth 7 on an Intel i9-14900K. The engine is available as a C++/CUDA library and as a Wolfram Language paclet.

Keywords: hypergraph rewriting, graph canonicalization, multiway systems, GPU computing, Wolfram Physics Project

1 Introduction

The Wolfram Physics Project [1] models the universe as an abstract rewrite system. Its states are hypergraphs, in which distance is path length through the edges, and its physics is given by the causal structure of the rewrites rather than by the data being rewritten. Multiway evolution applies every applicable rule at every match of every reachable state. It produces three structures: the multiway evolution graph of states and the events between them; the causal graph, which links an event to the events whose output edges it consumes; and the branchial graph, which links events that consume a common edge of the same state.

Exhaustive multiway evolution is expensive. Each step multiplies the number of histories by the frontier's match count, and a rule that adds edges gives each state more matches than its parent, so the count grows super-exponentially: on the canonical Wolfram rule the ratio between consecutive depths rises from 1.7 at depth two to 11.7 at depth seven. The existing Wolfram Language implementation takes 428 s to reach depth 7 of the corpus's `wolfram-2to4` rule.

Challenges. An efficient multiway engine must address four problems.

1. *Exact state identity.* Two states are the same node of the evolution graph if and only if their hypergraphs are isomorphic, so every new state must be tested against all previous ones up to isomorphism. No polynomial-time algorithm is known for this, and cheaper invariants merge non-isomorphic states.

2. *Irregular parallelism.* The frontier's width depends on the data and is not known in advance, work is created inside running tasks, and every state, event and relation is shared between workers.

3. *Determinism.* Under concurrency a state can be reached along a long path before a short one, and causal edges arrive in any order. The output must nonetheless be the same at every thread count.

4. *Scale.* The raw system grows super-exponentially, so deep evolution needs ways to compute less than the full system: expanding one state per isomorphism class, sampling, and continuing a partial run.

Contributions. We present a multiway hypergraph rewriting engine for CPU and GPU.

1. **Exact canonicalization.** Every state is canonicalized by McKay-style individualization–refinement (IR) [2], and the canonical hash is the deduplication key. We show that the saving from any cheaper filter in front of IR is bounded by the ratio of canonical classes to raw states, which falls toward zero with depth (Section 3).

2. **A lock-free, deterministic engine.** Matching follows HGMatch's match-by-hyperedge join [3], with matches forwarded from parent to child. All shared structures are lock-free, and idle workers wait on an address without using cycles. Depths are defined by shortest path and relaxed concurrently, and the causal graph's transitive reduction is maintained online, so the output does not depend on the schedule (Sections 4 and 5).

3. **A resident GPU kernel.** A CUDA kernel runs matching, rewriting and canonicalization for the whole evolution without returning to the host, and shares its decision procedures with the CPU engine as single-source code (Section 7).

4. **Exploration strategies.** Quotient exploration expands one state per isomorphism class and reconstructs every raw event and relation exactly; keyed sampling is reproducible at any thread count; and sessions continue a run from its frontier (Section 6).

5. **Verification and evaluation.** The concurrent data structures and protocols are model-checked under the RC11 memory model and in TLA+ (Section 9). The engine agrees exactly with a brute-force oracle on every corpus workload, and is 50,384× faster than `MultiwaySystem` at depth 7 (Section 8).

Section 2 defines the objects and reviews related work. Section 3 describes state and event identity, Section 4 the matcher, Section 5 the engine's architecture, Section 6 its exploration strategies, and Section 7 the GPU kernel. Section 8 evaluates the engine, Section 9 describes the verification of its concurrent code, and Section 10 concludes.

2 Background and Related Work

This section defines rules, matches, the multiway evolution graph and the causal and branchial relations, and reviews related systems.

2.1 Hypergraph Rewriting Systems

A *hypergraph* $\mathcal{H} = (V, E)$ consists of a vertex set V and a set of hyperedges E, where each hyperedge $e \in E$ is an ordered tuple of vertices $(v_1, v_2, \ldots, v_k)$ with $v_i \in V$. The Wolfram Physics Project uses hypergraphs because distance in them is path length, which gives rewriting a notion of locality without fixing a space in advance, as strings and cellular automata do.

Definition 2.1 (Rewriting Rule)**.** *A rewriting rule $\mathcal{R} = (L, R)$ consists of a left-hand side pattern L and right-hand side replacement R, both hypergraphs. A* match *of L in $\mathcal{H}$ is an injective mapping $\phi : V_L \to V_{\mathcal{H}}$ such that the image of each hyperedge in L appears in $\mathcal{H}$.*

Rules are applied in the single-pushout form: the matched image is deleted and the replacement glued in, without the gluing conditions of the double-pushout construction [4–6]. Applying rule $\mathcal{R}$ at match ϕ produces a new hypergraph $\mathcal{H}'$ by:

1. Removing all hyperedges in $\phi(L)$ from $\mathcal{H}$
2. Adding hyperedges from R, extending ϕ to map new vertices in R to fresh vertices

2.2 Multiway Evolution

Multiway evolution applies every applicable rule at every match of every state, producing a directed graph of states connected by events. Figure 1 shows two steps of a small rule in full: the raw states, the canonical states they reduce to, the events and the causal edges.

Definition 2.2 (Evolution Graph)**.** *Given initial state $\mathcal{S}_0$ and rule set $\{\mathcal{R}_i\}$, the evolution graph $G = (\mathcal{S}, \mathcal{E})$ contains:*

- *States $\mathcal{S}$: all hypergraphs reachable from $\mathcal{S}_0$*
- *Events $\mathcal{E}$: transitions (s, s', r, ϕ) where state s rewrites to s' via rule r at match ϕ*

The *causal graph* has an edge from event p to event c when c consumes an edge that p produced. The *branchial graph* has an edge between two events that consume a common edge of the same state; it records where the evolution branches, which the Wolfram Physics Project relates to quantum superposition [1].

2.3 The Graph Isomorphism Problem

Deduplicating states requires deciding whether two hypergraphs are isomorphic, that is, equal up to a relabeling of vertices. Graph isomorphism is in NP and is not known to be NP-complete or to be in P [7]; the best known algorithm is quasipolynomial [8]. This engine computes an exact canonical form for every state, which is equal for two states if and only if they are isomorphic, and deduplicates states by a 64-bit hash of that form (Section 3).

2.4 Related Work

Graph canonicalization: The nauty/Traces line of work [2] established individualization–refinement (IR) as the standard exact method for graph canonicalization. It is exponential in the worst case and fast on typical instances, and it is the canonicalizer this engine uses. Weisfeiler–Leman color refinement [9, 10] runs in polynomial time but gives the same value to some non-isomorphic graphs, so it cannot serve as an exact identity.

Hypergraph rewriting implementations: SetReplace [11] is the implementation most closely associated with the Wolfram Physics Project, and it computes a different object. Its multiway construction is a token–event graph over one growing pool of tokens: its vertices are tokens and events rather than states, a token consumed by one event remains available to others, and no state identity is computed. Its only isomorphism test during evolution is an optional deduplication of events with the same input tokens and isomorphic outputs. This engine enumerates the states graph under exact isomorphism, which requires canonicalizing every state as it is produced (Section 3), so canonicalization rather than matching dominates its cost. A wall-time comparison between the two would compare different computations. Section 8 therefore compares against two implementations of the same object: the `Wolfram/Multicomputation` paclet's `MultiwaySystem` [12][1], all of whose properties return Wolfram `Graph` objects, so its time includes building them; and this project's *reference implementation*, a direct transcription of the definition that returns data rather than graphs and serves as the correctness oracle throughout. *Reference implementation* always means the latter.

Subgraph matching on hypergraphs: HGMatch [3] matches one hyperedge at a time rather than one vertex at a time, seeding a join from one pattern hyperedge and extending it by one hyperedge per step; it restricts candidates by a label-derived signature, and runs as a dataflow on a work-stealing task scheduler. This engine's matcher uses all three. Hyperedges here are unlabelled, so the signature is the edge's vertex-repetition pattern (Definition 4.1). When an expansion has a variable already bound, its candidates come from the vertex index rather than the signature bucket; on the device this reduced match-kernel time on one workload from 944 ms to 4 ms.

GPU graph algorithms: GPUs have been applied to subgraph matching [13], graph mining [14] and general graph algorithms [15]. This engine's match kernel follows the warp-centric organization of G2Miner [16]: one block per (state, rule) pair, its threads dividing the seed candidates and each running its own depth-first search. Unlike these systems, the kernel stays resident across evolution steps, because each step's output states are the next step's inputs.

Dynamic graph algorithms: The causal graph's transitive reduction is maintained as edges are added. Goranci et al. [17] give fully dynamic transitive-reduction algorithms under insertions and deletions in $\mathcal{O}(m + n \log n)$ amortized time. This engine's case has insertions only, arriving in

[1] Distributed through the Wolfram Paclet Repository, https://resources.wolframcloud.com/PacletRepository/resources/Wolfram/Multicomputation/. Every comparison uses version 0.1.8, the latest published release (23 November 2024; checked 31 August 2026).

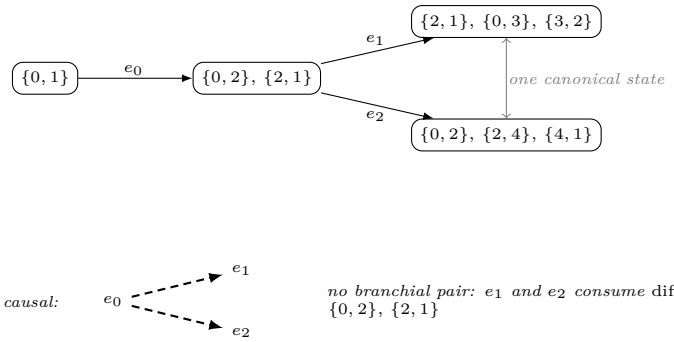


Figure 1: Two steps of $\{x, y\} \to \{x, z\}, \{z, y\}$ from a single edge, as the engine reports them. Four raw states reduce to three canonical states, because the two depth-2 results are isomorphic. Both causal edges come from e_0, because each depth-2 event consumes one of the two edges e_0 produced. The branchial relation is empty: e_1 and e_2 start from the same state but consume different edges. Every element is taken from the engine's output.

topological order, from concurrent workers, and the retained edge set must not depend on the schedule (Section 5.7).

Term rewriting: Term rewriting theory [18] treats confluence and termination for tree-structured terms, and fast implementations [19] optimize similar pattern matching. In hypergraph rewriting the objects are graphs, so matching is subgraph matching rather than tree matching, and states are compared up to isomorphism rather than syntactic equality.

3 State and Event Identity

Deduplication decides how large the explored set is: without it, every path to a state adds a separate vertex, and the number of paths grows super-exponentially when rewrites add edges. Two states are the same node when their hypergraphs are isomorphic, with vertex relabeling allowed and within-edge order and edge multiplicity preserved. The exact mode computes an individualization–refinement (IR) canonical form for every state and deduplicates by a 64-bit hash of that form, which two isomorphic states always share; two non-isomorphic states share it only through a hash collision, which the engine does not check for.

3.1 Individualization–Refinement

The canonical form is computed by McKay-style individualization–refinement [2], the family of algorithms behind nauty and Traces. IR produces a canonical labeling, together with automorphism generators and vertex and edge orbits, such that two hypergraphs receive the same canonical form if and only if they are isomorphic.

Definition 3.1 (Canonical Form)**.** *A canonicalization* Canon *maps each hypergraph to a labeled representative such that* $\textsc{Canon}(\mathcal{H}_1) = \textsc{Canon}(\mathcal{H}_2)$ *iff* $\mathcal{H}_1 \cong \mathcal{H}_2$.

IR refines a vertex coloring until it is stable. If a cell with more than one vertex remains, it *individualizes* one of its vertices (gives it a unique color) and recurses, which produces a search tree of refinements. Automorphisms found during the search prune symmetric branches, and the lexicographically extremal leaf defines the canonical labeling.

Remark 3.2 (Complexity)**.** *Like nauty, IR is exponential in the worst case and polynomial in the state size on states without large automorphism groups. The worst case occurs on highly symmetric states, where the exact mode's cost is concentrated. Proposition 3.3 bounds what a cheaper filter in front of IR can save.*

The automorphism generators and edge orbits from IR are used again for event canonicalization and for the exact reconstruction of causal and branchial multiplicities.

Search bounds. The search has two bounds: the individualization depth and the number of automorphism generators recorded. Exceeding either is reported, and the engine raises the bound and runs the search again. The two fail differently. Reaching the depth bound before the coloring is discrete produces no canonical form, so the failure is visible. A full generator table still produces a correct canonical hash, but the orbits, which are built by fusing edges over the recorded generators, come out too fine. The quotient reconstruction identifies instances by these orbits (Section 6.1), so a truncated generator table would give a wrong result with no error. The generator bound is therefore raised on overflow in the same way as the depth bound. Raising the generator bound cannot fix a depth failure, so escalation stops at the first depth failure.

Symmetric states. When a state has nontrivial automorphisms, IR selects one of several equivalent labelings by a tie-break within cells, and the tie-break depends on the order in which the state's vertices are numbered. Two numbering conventions give the same canonical hash but can assign different ranks to the same edge. Edge ranks identify consumed and produced edges across a rewrite, so one numbering convention is shared by every caller. On the equivalence probe's 4063-state corpus, encounter-order numbering and sorted-unique numbering assign different ranks on 103 states, all with nontrivial automorphisms.

3.2 A Ceiling on Pre-Filters

A sound invariant cheaper than IR, such as Weisfeiler–Leman color refinement [9, 10], could filter states before canonicalization: hash every state cheaply and run IR only when the cheap hash repeats. Proposition 3.3 bounds the saving.

Proposition 3.3 (Ceiling on a pre-filter)**.** *Let a run produce R raw states falling into C isomorphism classes, and let f be a sound isomorphism invariant: isomorphic states receive equal f-values. Consider deduplication that computes f on each raw state, and invokes IR only when the value has been seen before. Then the number of IR invocations is at least $R - C$, so the fraction of IR calls avoided is at most C/R, and one f-pass is paid on all R states.*

Proof. Soundness means states in one class share an f-value, so the number of distinct values is at most C. A

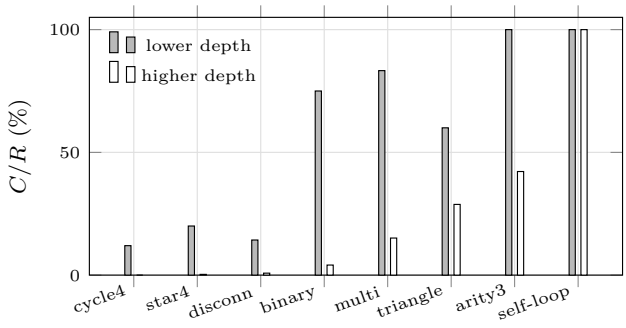


Figure 2: C/R for each corpus workload at two depths: the upper bound of Proposition 3.3 on the fraction of IR calls a pre-filter can skip. It falls with depth on every workload that keeps rewriting. `self-loop` reaches a fixed point after one event and stays at 100%. Values as in Table 1.

state whose value is already present cannot be resolved by f: equal values are consistent both with isomorphism and with an f-collision, so IR is invoked. At most one state per distinct value finds its value absent, hence at most C states skip IR and at least $R - C$ invoke IR. □

C/R depends on the rule and the depth. Table 1 and Figure 2 give it for each corpus workload at two depths.

Table 1: **Canonical classes as a fraction of raw states, per corpus workload.** C/R is the upper bound of Proposition 3.3 on the fraction of IR calls a pre-filter can skip. The event count is given beside it because a ratio from a small run is weaker evidence than the same ratio from a large one.

Case	Rule type	Events	C/R	C/R at lower depth
`cycle4-automorphic`	automorphism	68,184	0%	12%
`star4-automorphic`	automorphism	68,184	0.3%	20%
`disconnected-lhs`	disconnected	126	0.8%	14.3%
`binary-growth`	productive/arity2	873	4.1%	75%
`multi-rule`	multi-rule	237	15.1%	83.3%
`triangle`	productive/mixed-conn	124	28.8%	60%
`arity3-growth`	arity3	153	42.2%	100%
`self-loop`	self-loop/arity2	1	100%	100%

On every rule that keeps rewriting, C/R falls with depth, because multiway evolution reaches the same state along many histories and each repeat is a repeated filter value that IR must still resolve. The ratio stays high only on workloads that reach a fixed point within one or two steps, such as `self-loop` ($C/R = 100\%$ over two raw states and one event), and those have little canonicalization work to save.

The one exact shortcut, reusing a canonical form when two raw states have identical edge tuples, never applies: every rewrite assigns fresh vertex identifiers, so on `wpp` the number of distinct state contents equals the number of raw states at depths six and seven (3,867 and 45,317).

Quotient exploration and the event identities of Section 3.4 need automorphism data, and both take the edge orbits from the same IR pass.

3.3 State Canonicalization Modes

The engine's *state canonicalization mode* sets how states are identified. NONE keeps every raw state distinct. AUTOMATIC deduplicates by a hash of the edge tuples as given, which is not an isomorphism invariant: it identifies two states only when their edge tuples are equal, so isomorphic states with different vertex labels stay distinct. FULL is the exact mode: the IR canonical hash of every state is the deduplication key.

3.4 Event Identity

Events can be identified at three granularities:

ByEndpointStates: Two applications are the same event when they run between the same pair of canonical states. $\mathcal{O}(1)$ signature computation.

ByConsumedProducedEdges: The signature additionally includes the canonical ranks of the edges the application consumed and produced. $\mathcal{O}(E)$ signature computation.

DistinctApplications: No signature is computed and every application is its own event. This is the identity the raw causal graph is defined over.

Each refines the one before, so the event count is non-decreasing in that order; Table 16 gives it per workload. BYCONSUMEDPRODUCEDEDGES needs the correspondence between the edges of isomorphic states, which is the per-edge canonical rank from the same IR pass as the state's hash.

Independence from the state mode. Event identity is defined over isomorphism classes of states in every state mode. Outside the exact mode the state key is a content hash, which gives isomorphic states with different labels different keys, so event representatives are always resolved through the IR canonical hash when event canonicalization is on. The `mode_matrix_probe` gate checks that the event count is the same in every state mode. The canonical hash is computed only when event canonicalization is on, and the same IR pass gives both the hash and the edge ranks, so this costs one pass per state.

Configuration dependencies. The canonical-class event identity and quotient exploration both need edge orbits, which only the exact state mode computes. A run that requests either with another state mode builds the causal graph from raw edge identities instead and returns a warning naming the setting that is needed; the engine does not change the state mode itself.

4 Pattern Matching

Pattern matching finds every instance of a rule's left-hand side in a state, and it is the first step of every expansion. On the Wolfram rule, whose states have at most fourteen edges at depth six, matching, its join and the rewrite together account for 10.6% of executed instructions (Table 17); canonicalization (31.1%) and the quotient reconstruction (19.3%) are larger.

4.1 Edge Signatures

Definition 4.1 (Edge Signature)**.** *The* signature *of hyperedge* $e = (v_1, \ldots, v_k)$ *is the pattern* $\sigma(e) = (s_1, \ldots, s_k)$ *where* $s_i = s_j$ *iff* $v_i = v_j$*.*

For example, edge $(3, 3, 7)$ has signature $(0, 0, 1)$. A pattern edge can match a data edge only if their signatures are compatible. For a pattern signature with d distinct variables, the number of compatible data signatures is the Bell number B_d [20]: $1, 2, 5, 15, 52$ for $d = 1, \ldots, 5$. The host tests each candidate's signature against the pattern's directly; the device precomputes the compatible set for each pattern edge (Appendix B).

4.2 Candidate Edges by Ancestry

A state is a view into the shared edge pool (Section 5.2), and matching asks two questions of it: which of its edges contain a given vertex, once a pattern variable is bound, and which of its edges have a compatible signature, for the first pattern edge. Both are answered from the state's ancestry. Every edge of a state was added by exactly one state on its chain of ancestors: the root added all of its edges, and each later state added the edges its event produced. Each state indexes its own added edges by vertex once, when it is created, so the edges of S that contain v are

$$\bigcup_{A \in \text{chain}(S)} \{e \in \text{Contrib}(A)[v] : e \in S\}, \tag{1}$$

where $\text{Contrib}(A)[v]$ is A's sorted incidence list for v, and the test $e \in S$ excludes edges consumed by a later event on the chain. A query costs one search of a small sorted array per ancestor, nearly every edge it tests is in the state, a state's own index is the size of its event's right-hand side, and nothing is stored per edge. A global index over all edges of the evolution, filtered per state, would test every edge ever created at v; a state at depth seven holds under two percent of those.

4.3 Matching Algorithm

Algorithm 1 gives the task-based matcher.

Algorithm 1 Task-Based Pattern Matching

```
1:  procedure FindMatches(L, H)
2:      e_1, ..., e_k ← MatchOrder(L)        ▷ connected: no step is a product
3:      Enqueue ScanTask(e_1)
4:      while task queue non-empty do
5:          τ ← Dequeue()
6:          if τ is ScanTask(e_1) then
7:              for all e ∈ H with σ(e) compatible with σ(e_1) do
8:                  φ ← binding of e_1 to e
9:                  Enqueue ExpandTask(2, φ)
10:             end for
11:         else if τ is ExpandTask(i, φ) then
12:             if i > k then
13:                 yield complete match φ
14:             else
15:                 for all e ∈ Candidates(e_i, φ) not in φ do
16:                     if e is consistent with φ on e_i then
17:                         Enqueue ExpandTask(i+1, φ ∪ {e_i ↦ e})
18:                     end if
19:                 end for
20:             end if
21:         end if
22:     end while
23: end procedure
```

The match order $e_1, \ldots, e_k$ is fixed when the rule is added, and it is connected: every edge after the first shares a variable with an earlier one, so no step of the join is a cartesian product, and $\text{Candidates}(e_i, \phi)$ is the incidence list of a vertex of e_i already bound in ϕ (Section 4.2). The host runs the join in two forms: a depth-first join that runs a whole search inside one task and submits a job only for each completed match, and the task form above, which submits a task per candidate so that work stealing can divide the search. Both forms, and the device, share one implementation of the recursion, the edge-injectivity check, binding and unbinding, and the choice of the next pattern edge.

4.4 Relation to Worst-Case-Optimal Joins

Treated as a conjunctive query, a left-hand side has an AGM bound [21]: the maximum number of results it can have on a database of a given size. A join is worst-case optimal if its running time matches that bound up to logarithmic factors [22]. This engine's edge-at-a-time join meets the bound on left-hand sides of one or two hyperedges, which covers every rule in the oracle corpus of Section 8; the sweep of Section 8.6 also covers three and four edges. It does not meet the bound in general: on a cyclic left-hand side of three hyperedges the bound is $N^{1.5}$ for a state of N edges, while an edge-at-a-time join performs $\Omega(N^2)$ work, because two of the three edges can be bound before the third constrains them, whatever the join order. The static analysis of Section 5.4 records for each rule whether its left-hand side is acyclic and its integral edge cover, and the engine warns when a rule's left-hand side is cyclic over three or more edges.

4.5 Match Forwarding

When state s' is derived from s, a match of s is still a match of s' unless it used an edge the rewrite removed. The engine forwards such matches from parent to child instead of finding them again:

Proposition 4.2 (Match Forwarding)**.** *Let $\mathcal{M}$ be a match in state s, and let s' be derived from s by applying rule r at match $\mathcal{M}'$. Then $\mathcal{M}$ remains valid in s' iff edges$(\mathcal{M}) \cap$ edges$(\mathcal{M}') = \emptyset$.*

A child receives its parent's matches once, after the parent's matching has finished. Each state counts its outstanding match tasks and *drains* when the count reaches zero. At its drain a parent hands every registered child the stored matches that use none of the edges the child's rewrite consumed, and a child registered after the drain takes them at registration. The two sides meet through a sequentially consistent fence on each side, and a claim on the child makes the second delivery a no-op. The child counts its inheritance as one of its own tasks, so it drains only once its inherited matches are stored, and its own children then inherit a complete list from it. No state reads a list above its parent's.

Forwarded matches are collected per parent and delivered to the child together. The static analysis of Section 5.4 turns forwarding on for rule sets whose left-hand side is a join, and a run with a per-state match cap turns it off (Section 6.2); Section 8 reports the ablation.

4.6 Delta Matching

A match present in the child but not in the parent must use at least one edge the rewrite produced, because a match built only from surviving edges was already a match in the parent. The child's own scan therefore looks only for matches that contain a produced edge, and a rewrite produces only the right-hand side's edges, however large the state is.

Forwarding and the delta scan divide the child's matches between them: forwarding supplies the matches that use no produced edge, and the delta scan the matches that use at least one. A match that uses k produced edges is found k times by the delta scan, once from each, so matches are deduplicated by content, and the duplicate check runs before a match record is allocated.

A validation mode checks this division: it also computes every state's matches by a full scan and compares them with what forwarding and the delta scan supplied. A missing match is attributed to forwarding if it uses only surviving edges and to the delta scan otherwise. The validator also counts how many comparisons it made, so a mismatch count of zero is known to cover a non-zero number of states.

5 System Architecture

Two rules constrain the design. First, every decision procedure (an ordering, an identity, a hash, a predicate) has one implementation, compiled for both the host and the device. Second, no thread waits for another: every shared structure is lock-free, and where threads must agree, one compare-exchange decides and the losers adopt the winner's result.

Algorithm 2 gives the evolution loop. A match task for each new state runs the join of Algorithm 1 and creates a rewrite task for each accepted match. A rewrite task applies its match, claims the child's canonical identity, records the event and its causal and branchial relations, forwards the surviving matches (Section 4.5), and creates a match task for the child if it was first to claim the child's identity. Under a per-state cap or sampling, the choice among a state's matches waits until the state's matching is complete (Section 6.2), and under quotient exploration only one state per class gets a match task (Section 6.1).

Algorithm 2 Task-Based Multiway Evolution

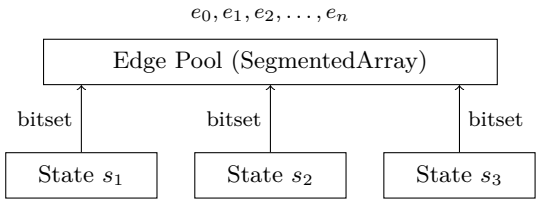
```
1:  procedure EVOLVE(L, S_0, depth D)
2:      for all s_0 ∈ S_0 do
3:          publish s_0; enqueue MATCHTASK(s_0)
4:      end for
5:      while tasks pend at a depth ≤ D do
6:          τ ← DEQUEUE()
7:          if τ = MATCHTASK(s) then
8:              M ← inherited ∪ FINDMATCHES(L, s)
9:              for all deduplicated m ∈ M do
10:                 enqueue REWRITETASK(s, m)
11:             end for
12:         else if τ = REWRITETASK(s, m) then
13:             apply m, creating raw child c
14:             canonicalize c; claim its identity
15:             record event, causal and branchial edges
16:             forward s's surviving matches to c
17:             if claim won and depth(c) < D then
18:                 enqueue MATCHTASK(c)
19:             end if
20:         end if
21:     end while
22: end procedure
```

5.1 Shared Host and Device Implementation

The CPU engine and the CUDA port have separate schedulers and share their semantics. Each decision procedure they share is one function compiled for host and device from a single source: the join that enumerates matches, the edge-signature rule, the rewrite rule that determines a produced edge's vertices, the event identity, the content-ordered state key, the individualization–refinement canonical hash, and the quotient-causal dynamic program and its per-instance replay. There are eleven such shared functions; the other three are the matcher's task core, the sampling draw and the slot rule. The differential tests between host and device therefore cover what differs between them: scheduling, memory placement and queueing.

5.2 Unified Hypergraph Storage

All edges are stored in one shared pool, and each state is a bitset over that pool (Figure 3), so a child that differs from its parent by a few edges stores only the difference.



States are views (SparseBitset) over shared edges

Figure 3: Unified edge storage with state views

The bitset is divided into chunks, and a child shares its parent's chunks until it changes one, at which point that chunk is copied. A rewrite that changes a few edges allocates a few chunks, and shared chunks are never modified, so readers of the parent are unaffected by the child. Table 3 gives the arena bytes of each corpus workload. Appendix C specifies the lock-free structures, their per-operation costs, the memory model, and the behavior at configured limits.

5.3 Allocation

Evolution allocates from per-worker arenas. Each worker bumps a pointer into blocks it owns, so allocation during matching and rewriting needs no synchronization, and memory is freed in bulk by resetting the arena. The global allocator is called only to obtain those blocks: Table 3 reports 259 to 280 calls for an entire run, which on the largest workloads is 0.004 calls per event.

5.4 Static Rule Analysis

When a rule is added, the engine computes properties that follow from its patterns alone and uses them to choose a strategy. For each rule it records the difference in edge count between the two sides, which says whether states grow; the number of new vertices the right-hand side introduces; and the left-hand side's arities. Treating the left-hand side as a conjunctive query, it computes whether the query is acyclic and an integral edge cover ρ, which bounds the matches in a state of N edges by N^{ρ}. The bound is tight when the query is acyclic, and is used only as an upper bound otherwise.

Each predicate is sound in one direction. The branching test is an over-approximation: a negative answer means no two matches can ever share a consumed edge, and a positive answer means only that this has not been ruled out. The engine acts only on the negative answer: the branchial relation is then empty for every initial state, and the engine returns it empty without computing it. No analysis here decides termination, finiteness of the reachable set, or confluence, which are undecidable for rewriting systems of this kind.

5.5 Determinism Contract

The state set, the event set and the causal and branchial relations are a deterministic function of the rules, the initial state, the step count and the options. They do not depend on the number of worker threads, the order in which tasks run, or whether quotient exploration is enabled. State and event identifiers are allocated in arrival order and vary between runs; the observables are defined over canonical identities. A determinism gate checks this across thread counts and seeds: it fingerprints the expanded state set,

the multiset of (input, output, rule) transitions, and the causal and branchial relations separately, and reports which component differs and under which configuration. The components are compared separately because the causal relation can differ by a single edge while the others agree.

The count caps MaxStatesPerStep and MaxSuccessorStatesPerParent are outside the contract. They admit states until a counter reaches the cap, so which states are admitted depends on arrival order and varies with the worker count. Making them schedule-independent would require holding every candidate at a depth until the depth is complete, which is a barrier. A capped run guarantees the count at every worker count, not which states make it up.

5.6 Depth Semantics and Termination

The step budget applies to the *shortest* path from an initial state to a canonical state, not to the path along which the state was first reached. Under concurrency a state can be reached first along a long path and later along a short one, so a budget on arrival depth would include or exclude states according to which path won the race, and the explored set would vary with the thread count.

The engine therefore *relaxes* depths: when a state is reached along a path shorter than its recorded depth, the recorded depth is lowered and the change is propagated to its children, and from them onward. This terminates because each accepted change lowers a depth, which is bounded below by zero. A state is matched at most once however often its depth is lowered, because the claim that admits a state to expansion is separate from its depth.

Relaxation races the registration of a new child. Registration publishes the child and then reads the parent's depth; relaxation lowers the depth and then scans the child list. Sequentially consistent fences on both sides rule out the case where the scan misses the child and the read misses the lowered depth, so a child always sees its parent's current minimum depth. A state past the budget stays on the frontier without being claimed, so a shorter path found later can still bring it within the budget. A state is claimed against the same bound its match task is admitted under, so a claimed state is never left unmatched.

Work budgets are taken by compare-and-swap. An increment followed by a rollback would briefly publish a count above the limit, and a thread reading the count at that moment would prune work that another schedule keeps.

Termination is detected without a barrier: a depth is complete when it has no live work and the depth below it is complete. This suffices because a task at depth d only creates work at depths above d, so once depth $d-1$ is complete nothing new can arrive at d.

5.7 Transitive Reduction of the Causal Graph

The causal graph has an edge for every dependency between events, including those implied by longer paths. Its transitive reduction removes an edge $p \to c$ whenever a path $p \rightsquigarrow c$ of length greater than one exists; on an acyclic graph the reduction is unique.

Goranci et al. [17] give the first fully dynamic algorithms for the transitive reduction of a general directed graph under insertions and deletions, in $\mathcal{O}(m + n \log n)$ amortized update time, and $\mathcal{O}(m + n^{1.585})$ worst case with fast matrix multiplication. This engine's case is simpler in two ways. First, causal edges are only inserted, so an edge dropped as implied never needs to be reconsidered. Second, edges arrive in topological order, because an event's producers exist before the event, so when an edge (p, c) arrives the closure below p is final. The procedure below is the specialization of the problem to this case.

Online maintenance. The reduction is kept up to date after every insertion, because a session (Section 6.3) can be queried between steps, and because building the full relation first would allocate the transitive closure that the reduction avoids.

Procedure (Algorithm 3). Let $e = (p, c)$ be a new causal edge: event p produced an edge that event c consumed. If c is already reachable from p through retained edges, e is implied and dropped; otherwise it is retained. A retained edge never needs to be removed later. A consumer's incoming edges are added by its own worker in descending producer order, and an ancestor's id is always smaller than its descendant's. A retained (p', c) could become implied only through a path $p' \to \cdots \to x \to c$. If (x, c) arrived first, the query for (p', c) saw the path and dropped the edge; if (x, c) arrived later, then $x < p'$ by the descending order, while reachability from p' requires $p' < x$, a contradiction. Hence deciding each edge once on arrival gives the exact reduction. The descending order also makes queries cheap, because the closure from nearer producers is built before farther ones are tested, and it needs no sorting, because events are numbered in creation order.

Algorithm 3 Online Transitive Reduction (insertion-only)

```
1: procedure ConsumeEdges(event c, producers P)
2:     for all p ∈ P in descending event id do
3:         if c reachable from p via retained edges then
4:             drop (p, c) as implied
5:         else
6:             retain (p, c)
7:         end if
8:     end for
9: end procedure
```

Order independence. The retained edge set must depend only on the causal DAG, not on the order in which concurrent workers insert edges; the determinism gate checks this across thread counts and seeds. Full capture meets it through the arrival order above: each consumer's edges are added by its own worker, in descending producer order, after its producers' edges are complete, and under that order the decision on arrival is exact however consumers interleave. The quotient replay meets it the same way: an event's incoming edges are added in its own application, after its producers' applications, and all at once, so a producer is dropped exactly when it is an ancestor of another producer of the same event, found by one backward search from all of them. Under quotient exploration without the replay, the relation is kept between canonical event ids, which are not ordered this way, so it is stored unreduced and reduced when it is read; a finite DAG has one transitive reduction, so the result depends only on the stored edges.

Only the causal graph is reduced. States, events and the branchial relation are not.

6 Exploration Strategies

Exhaustive evolution becomes infeasible at moderate depth, and a caller may need only part of it. The engine has four independent ways to limit an exploration, and they can be combined:

- **Caps.** A total state count, a total event count, a count of states admitted per step (MaxStatesPerStep) and a count of successors per parent (MaxSuccessorStatesPerParent). Each is an atomic counter, so the count retained is exact at every worker count; which states fill a cap depends on arrival order (Section 5.5). A run that reaches a cap returns what it computed with a warning (Section C.3).
- **Sampling.** Per-rule transition rates and a per-state match cap keep a subset of transitions chosen by keyed pseudorandom draws, so the sample is reproducible for a seed at every worker count (Section 6.2).
- **Quotient exploration** (Section 6.1) expands one state per isomorphism class and reconstructs the raw relations exactly.
- **Steering.** A session (Section 6.3) reports the frontier after each step, and the next step can be restricted to part of it; the rest stays on the frontier.

Caps and steering compute a subset exactly; sampling computes a random subset; quotient exploration computes the full result and expands less. The step budget bounds all four, and the reported frontier records where each stopped.

6.1 Quotient Exploration

In full expansion every raw state is expanded, so a state reached along k histories is matched k times, although canonicalization has already shown the repeats find no new states. Quotient exploration expands only one state per isomorphism class, at the shallowest depth the class was reached. Every equivalent state is still created and its incoming event recorded. Table 14 gives events produced, causal edges and wall time for both modes by depth. Both modes give the same canonical state set. The causal and branchial relations are defined over raw events, which quotient exploration does not all produce, so they are reconstructed.

The number of raw events is recovered by a forward summation. Under exact identity the canonical states graph has cycles, because a canonical state recurs at several depths, but its unrolling by depth is acyclic, and the multiplicity of each (canonical state, depth) pair satisfies

$$\begin{aligned} \mathrm{mult}[s_0, 0] &= 1, \\ \mathrm{mult}[c, k{+}1] \mathrel{+}= \mathrm{mult}[s, k] &\cdot w(s \to c), \end{aligned} \tag{2}$$

summed over the transitions into c; the raw event count is $\sum \mathrm{mult}$. This takes time polynomial in the number of (canonical state, depth) pairs.

The raw events and relations themselves are recovered by replaying each class's expansion against every instance of the class. The expansion is recorded once, in the *frame* of the first raw state that reached the class: each match found on that state is stored as indices into the frame's edges. Another raw state of the class is aligned onto the frame by the canonical edge ranks from its own IR run; this alignment is an isomorphism, defined up to an automorphism, so it maps matches to matches. The edge orbits from the same IR pass distinguish edge roles within a class (Section 3.1), so no further canonicalization is needed. An *instance* of a class at depth k is a vector holding one producer event per frame slot. Applying a recorded match to an instance creates one raw event; adds a causal edge from the producer of each consumed slot, in descending producer order so that the online reduction of Section 5.7 stays exact; adds a branchial pair with each earlier application at the same instance whose consumed slots overlap; and creates a child instance at depth $k+1$, in which surviving slots keep their producer and produced slots take the new event. Instances and recorded matches arrive concurrently, so each new instance replays every match already recorded for its class, each new match is replayed against every instance already present, and each (instance, match) pair is claimed exactly once. A replayed event has two identities: the triple (source class, target class, rule), which is an isomorphism invariant used for comparisons across runs and engines, and the run's event identity.

The reconstruction creates as many instances as the full system has raw states. On a two-rule set whose canonical state count grows linearly with depth (three to eight states at depths two to seven), the raw system has 146,599 states at depth six, and the reconstruction's cost follows that count. There, quotient exploration visited 49 raw states against full capture's 146,599 and ran 65 times faster.

The reconstruction runs only when the causal relation, the branchial relation or the raw event set is requested; a run that asks for canonical states alone skips it.

6.2 Sampled Exploration

The engine has two samplers. The *transition rate* keeps each candidate transition independently with a fixed probability, multiplied by a per-rule weight; the number of children a state keeps is then a random variable with the thinned distribution of the full one. The *per-state match cap* keeps at most k of each state's matches for each rule. Both choose by a pseudorandom value computed from the transition's canonical identity and the seed, so the same seed gives the same sample at any thread count (Section 5.5).

Choosing the rate. A fixed rate compounds with depth: a path of length L survives with probability q^L. On the workloads measured, rates of $1/8$ and $1/4$ end the evolution within the first step and $1/2$ leaves it growing, so the usable range is narrow and depends on the rule's branching factor. While a rate or weight below one is in effect, a state whose own draws all fail keeps its lowest-ranked own transition, so the evolution does not die out; a state with no matches of its own has no such fallback.

The per-state cap. Samples from different seeds do not add up to the exhaustive system, because a transition can be drawn only if its source state was created, which required an earlier draw to succeed. The per-state cap keeps at least one transition at every state it reaches, so each seed covers a connected skeleton, and different seeds keep different skeletons. The cap chooses k of a state's M matches by rank once the state's matching is complete, because choosing k of M needs all M; choosing by arrival order would make the result depend on the schedule. A match forwarded from an ancestor can arrive after the state's matching is complete, so a run under the cap turns match forwarding off and each state finds all of its matches itself.

Per-state and per-transition draws. Under full capture each raw state is created by exactly one event, so a draw per new state and a draw per transition are the same draw. Under quotient exploration a canonical state has many incoming transitions, and the two differ.

No barrier. A decision to keep a match is final once the match is rewritten, so a sample must be complete before

its members are rewritten. A sample drawn from a whole evolution step would be complete only when every worker finished the step, which is a global barrier. The engine therefore samples only over populations that complete locally: a state's matches for one rule are complete when that state's matching tasks have all finished, which the engine detects without looking at any other state.

6.3 Sessions

Evolution is also available as a session: it is opened on a rule set and an initial state, advanced by a number of steps, queried for any derived structure, and closed. The frontier is kept between steps, so each step continues the existing evolution instead of starting again.

The frontier is stored separately from the canonical identity map. The map records which states have been seen; the frontier records the states and pending rewrites that the step budget stopped, together with the depth each waits at.

Every session reply reports the frontier, and a step may name part of it. The named entries are advanced and the others stay on the frontier. A step that names no entries advances the whole evolution to the new total depth, and gives the same result as a single run to that depth. A step that names entries advances each of them by the requested number of steps from the depth it waits at, so entries at different depths each advance by the same amount. Naming a state that is not on the frontier is an error. A continuable run stopped by a state or event limit keeps the work the limit cut short on the frontier (Section C.3). Both devices implement this interface.

7 GPU Acceleration

Multiway evolution is an irregular workload for a GPU. The frontier's width depends on the data and is not known until the step that produces it, and every new state must be canonicalized before it can be compared with others, by the same IR code the host runs, whose control flow depends on the data. The kernel spreads IR's loops across the lanes of a warp where the loops allow it. A kernel launch per step would put a host round trip between every stage, so the engine runs one resident kernel that keeps the frontier on the device for the whole evolution. Appendix B specifies the device canonicalization, matching, work queues, termination and memory management.

7.1 Resident Kernel

A kernel launch has a fixed cost, which dominates the early steps of an evolution, where the frontier is small. The engine runs a single persistent kernel for the whole evolution, with the frontier, the match queues and the epoch counter kept in device memory, so a step costs a device-side barrier rather than a launch. The match, rewrite and state queues are resident, and a new state becomes a match task without returning to the host. The resident kernel is 24.4× faster than a per-step `evolve()` at depth four and 20.3× faster at depth eight (Section 8).

One block matches one (state, rule) pair, and its threads divide the candidates for the first pattern edge. Later pattern edges take their candidates from a vertex index through a variable that is already bound, as described in Appendix B.

7.2 Design Constraints

Nothing crosses the host–device boundary while the kernel runs. Rules and the initial state are uploaded and the queues seeded before the launch, and results are read after it; in between, the device decides what work exists, which worker takes it, and when the evolution has finished. Any round trip during the run would cost as much as the per-step round trip the design removes.

This has two consequences.

The kernel cannot grow a pool. Growing a pool needs a host allocation. When a pool overflows, the kernel records the overflow and returns the work completed so far; the host then re-runs the evolution with doubled capacities, up to eight times, and returns the last partial result with a warning if it still overflows (Section C.3).

Canonicalizer scratch cannot be sized from a batch. The resident kernel has no batch: states arrive continuously and the largest is not known until the run ends. Scratch pre-sized from a configured maximum, with a coarser hash above it, would let two non-isomorphic states share a key whenever one was too large to canonicalize exactly. Instead each state takes scratch from a device-side arena, sized from its own vertex, edge and occurrence counts. Exhausting the arena is a capacity overflow, handled as above; the exact path never falls back to a coarser identity.

8 Experimental Evaluation

The evaluation answers five questions.

Q1 (correctness). Is state identity exact (two states identified if and only if they are isomorphic) on every corpus workload, checked against an independent oracle? (Table 2)

Q2 (speed against the reference). How does evolution time compare with the Wolfram implementation of the same computation, and how does the ratio change with depth? (Table 4, Figure 4)

Q3 (parallel scaling). How does evolution time change with the worker count, does that depend on the rule, and what limits it? (Tables 5, 6, 7; Figure 5)

Q4 (device). Where is the GPU faster than the CPU, and at what grid size does it saturate? (Tables 9, 10, 12)

Q5 (attribution). Which parts of the engine account for its speed, and does any of them change the result? (Tables 15, 14, 16)

8.1 Experimental Setup

Hardware: Intel i9-14900K (8 performance cores with SMT and 16 efficiency cores, 32 threads, one L3), 31 GB RAM, NVIDIA RTX 4090 (24 GB, compute capability 8.9), under WSL2. Workers are placed in the engine's default order, read from the cache topology at start-up. The left-hand-side sweep (Figures 7 and 8, Table 8), the occupancy table (Table 13) and the hardware-counter measurements of Section 8.7 were taken on an AMD EPYC 9174F (16 cores, 32 threads, eight L3 domains), 503 GB RAM, with the same GPU model, on bare metal.

Software: Linux 6.18 (WSL2), GCC 15.2 with C++20, CUDA 13.3, Wolfram Language 14.3.

Workloads: A corpus of 17 workloads covering rule types (single and multiple rules; productive, idempotent, reductive and mixed-arity rules; several initial-state shapes), and deeper runs of the canonical Wolfram rule $\{\{x, y, z\}\} \to \{\{x, y, w\}, \{y, z, w\}\}$ for the scaling and GPU measurements.

Methodology: Wall times are medians over repeated runs on an otherwise idle machine, and the two arms of any comparison are measured in the same session, because run-to-run variation on this machine exceeds many of the effects reported. Counts, instruction counts and `getrusage` figures are deterministic and come from a single run. The left-hand-side sweep of Figure 7 reports the *minimum* of its repeats, because interference from other load can only slow a run, and it starts a timed run only when the machine is idle.

8.2 Correctness and Memory Footprint

Every corpus workload is evolved single-threaded in the exact mode to the depth listed with it, chosen so that the brute-force check stays tractable; depths differ between rows, so counts are comparable within a column but not across rows. Its canonical state set, event set, causal relation and branchial relation are compared with the reference implementation (Table 2), and each state set is checked against a brute-force isomorphism oracle. All seventeen workloads agree exactly.

Table 3 gives the memory the same runs use. Arena bytes come from the arena's own counter, which is deterministic, rather than from the resident set size. The global allocator is called 259 to 280 times per run, for set-up that does not grow with the evolution, so calls per event fall from 259 on the smallest workload to 0.004 on the largest.

Table 2: Canonical states, events, causal edges and branchial edges for each corpus workload; the brute-force oracle confirms every workload.

Workload	Class	Canon. states	Events	Causal	Branchial
binary-growth	productive/arity2	36	873	872	0
wolfram-2to4	productive/arity2	45	126	184	111
triangle	productive/mixed-conn	36	124	192	249
reductive-2to1	reductive/arity2	4	15	12	5
idempotent-2to2	idempotent/arity2	1	6	10	0
self-loop	self-loop/arity2	2	1	0	0
mixed-arity	mixed-arity	7	6	5	0
arity3-growth	arity3	65	153	152	0
disconnected-lhs	disconnected	1	126	248	63
arity4-growth	arity4	23	33	32	0
mixed-arity-lhs	mixed-arity/join	2	1	0	0
mixed-arity-init	mixed-arity/init	4	4	0	0
arity3-to-2	arity-reductive	4	4	0	0
arity1-with-binary	arity1/mixed	2	1	0	0
cycle4-automorphic	automorphism	7	68184	109992	68184
star4-automorphic	automorphism	201	68184	56360	0
multi-rule	multi-rule	36	237	214	108

Table 3: Arena bytes, heap bytes, and global `operator new` calls during evolution, per corpus workload, ordered by work performed.

Workload	Canon. states	Events	Arena B	Heap B	Heap allocs	Allocs per event
self-loop	2	1	373712	536120	259	259
mixed-arity-lhs	2	1	390488	536164	260	260
arity1-with-binary	2	1	390520	536152	260	260
mixed-arity-init	4	4	486544	536300	262	65.5
arity3-to-2	4	4	486668	536256	261	65.2
idempotent-2to2	1	6	476768	536144	260	43.3
mixed-arity	7	6	426256	536436	262	43.7
reductive-2to1	4	15	569352	536204	261	17.4
arity4-growth	23	33	587616	536424	261	7.91
triangle	36	124	1388464	537876	263	2.12
wolfram-2to4	45	126	1340752	538104	264	2.1
disconnected-lhs	1	126	1285632	536438	262	2.08
arity3-growth	65	153	1274464	537984	263	1.72
multi-rule	36	237	1536608	539580	266	1.12
binary-growth	36	873	2609332	549960	267	0.306
cycle4-automorphic	7	68184	125778300	1645548	280	0.00411
star4-automorphic	201	68184	93801448	1956060	280	0.00411

8.3 Comparison with MultiwaySystem

The timed comparison is against the `Wolfram/Multicomputation` paclet's `MultiwaySystem`, the published implementation of the same computation. The reference implementation (Section 2) checks that all three agree on state and causal-edge counts at every depth, and they do; its time is not reported. From depth four to seven the engine's core is 1,696× to 50,384× faster than

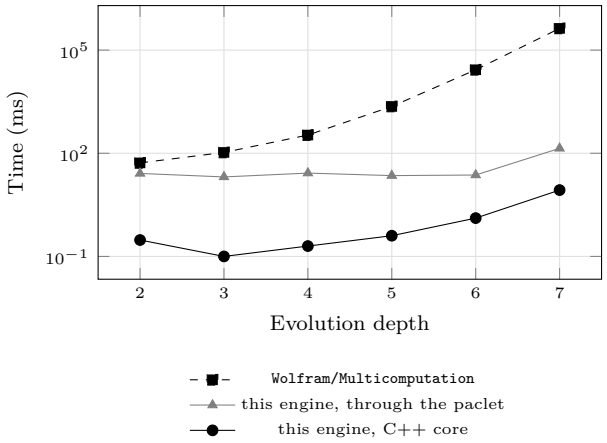


Figure 4: Evolution time against depth for `MultiwaySystem`, the engine through the paclet, and the engine's C++ core, on the same rule and initial state; all three give the same state and causal-edge counts at every depth. The gap between the first and last series is the speedup of Table 4; the gap between the last two is the cost of the Wolfram interface.

`MultiwaySystem`, and the ratio increases with each depth. The depth-two time includes `MultiwaySystem`'s one-time initialization, and at depths two to five the engine takes a fraction of a millisecond, close to the timer's resolution, so the ratios at those depths are lower bounds. At depth seven the engine takes 8.5 ms and `MultiwaySystem` 428 s, both well above the resolution.

Each side is timed through its user-facing interface. Every `MultiwaySystem` property returns a Wolfram `Graph` (`StatesGraph`, `CausalGraph`, `EvolutionGraph` and others), so its time includes building graphs. The engine is timed twice: through its C++ API, and through the paclet, which serializes the request to the engine's process over WXF, evolves, and builds the same kind of Wolfram structures on return. The difference between the two is the cost of the Wolfram interface on the same computation: at depth 7 it turns 8.5 ms into 0.14 s, a factor of 16×. Through the paclet the engine is still 3081× faster than `MultiwaySystem`.

8.4 Thread Scaling

Strong scaling is measured on the Wolfram rule at depths five to seven, the depths of Table 9, at 1, 2, 4, 8, 16, 24 and 32 workers; the host has eight performance cores with SMT and sixteen efficiency cores. At depth seven wall time falls at every step, by less each time, reaching a speedup of 10.2× at 32 workers (Table 5). At depths five and six, below 10 ms of single-threaded work, it stops falling at eight workers. The expanded state set and the transition multiset are the same at every thread count and seed. Table 6 gives the spread across rule shapes, and Table 7 what the extra time at high thread counts is spent on.

8.5 Parallel Efficiency by Rule Shape

The amount of parallel work in a run depends on the rule. Table 6 measures two rules at every thread count: a one-edge left-hand side, which matches by an index scan and expands each state narrowly, and a two-edge left-hand side, which matches by a join and produces a wider frontier. With both sized to seconds of single-threaded work, their efficiencies are within ten points of each other at every thread count (the lower of the two is 0.65 at eight threads and 0.43 at sixteen). The two-edge left-hand side still gains in wall time at thirty-two threads; the one-edge left-hand

Table 4: Wall time by depth against `MultiwaySystem` on `wolfram-2to4`: $\{\{1,2\},\{2,3\}\} \to \{\{1,2\},\{2,4\},\{4,3\}\}$ from $\{\{1,2\},\{2,3\}\}$. The engine is timed through its C++ API and through the paclet (WXF and Wolfram structure construction included); the speedup column divides `MultiwaySystem`'s time by the C++ core's. The reference implementation checks the counts and is not timed.

Depth	States	`MultiwaySystem` ms	Engine ms (paclet)	Engine ms (C++ core)	Speedup (core vs. `MultiwaySystem`)
2	3	52.5	25.9	0.3	175×
3	4	105.7	20.5	0.1	1057×
4	5	339.1	26.5	0.2	1696×
5	6	2298.6	22.4	0.4	5746×
6	7	26830.7	23.3	1.3	20639×
7	8	428264.0	139.0	8.5	50384×

Table 5: Strong scaling: median wall time and parallel efficiency by worker count, Wolfram rule at depths five to seven (28,542 canonical and 45,317 raw states at depth seven, at every worker count), workers in the engine's default placement.

Threads	depth 5		depth 6		depth 7	
	ms	eff.	ms	eff.	ms	eff.
1	0.9	1.00	9.0	1.00	127.3	1.00
2	0.5	0.79	5.1	0.88	70.9	0.90
4	0.5	0.48	2.9	0.76	40.6	0.78
8	0.4	0.28	1.8	0.63	23.5	0.68
16	0.4	0.13	1.8	0.31	15.8	0.50
24	0.4	0.09	1.7	0.21	12.5	0.43
32	0.4	0.06	1.7	0.17	12.4	0.32

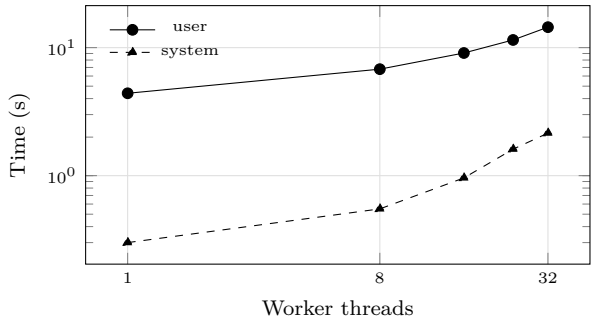


Figure 5: User and system time against worker count, for the runs of Table 7: across a 32× increase in workers, user time grows 3.3× and system time, seven to fifteen times smaller, grows 7×.

side is fastest at twenty-four. The decline is not thread start-up, since both single-threaded runs take seconds. Section 8.6 measures the same size axis at four sizes and greater depths, together with two other properties of the left-hand side.

Table 6: Wall time and parallel efficiency by thread count for a one-edge and a two-edge left-hand side.

Threads	one-edge LHS		two-edge LHS	
	ms	eff.	ms	eff.
1	4190.9	1.00	3464.6	1.00
2	2447.7	0.86	1844.9	0.94
4	1567.3	0.67	1120.5	0.77
8	806.7	0.65	671.1	0.65
16	579.8	0.45	509.4	0.43
24	500.8	0.35	466.1	0.31
32	515.5	0.25	451.5	0.24

8.6 Left-Hand-Side Properties

A left-hand side has three independent properties, each of which changes the matching problem:

- **Size**: the number of edges to bind. Each edge adds a factor to the search and a constraint that prunes it.
- **Arity**: the number of vertices per edge, which sets how much of the state one bound edge determines.
- **Connectivity**: how the edges share vertices. A path shares one vertex between consecutive edges; a star shares one vertex among all of them; a ring has no end; and a left-hand side with components that share no vertex is matched as a cartesian product, quadratic in the state size for two components.

Figure 6 gives the wall times of the sweep's ten timed workloads at every thread count, each at the deepest depth whose single-threaded run fits the sweep's time budget, and Figure 7 gives the same measurements as parallel efficiency along each property. Every curve in Figure 6 falls through 32 threads, mostly below sixteen: going from one thread to eight divides wall time by 2.3 (the star) to 4.5 (the mixed-arity chain), and going from sixteen threads to 32 reduces it by a further 4.6 to 23.7 percent.

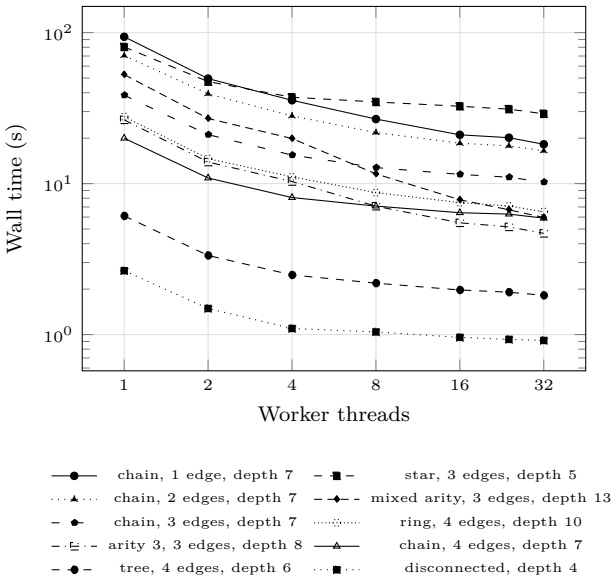
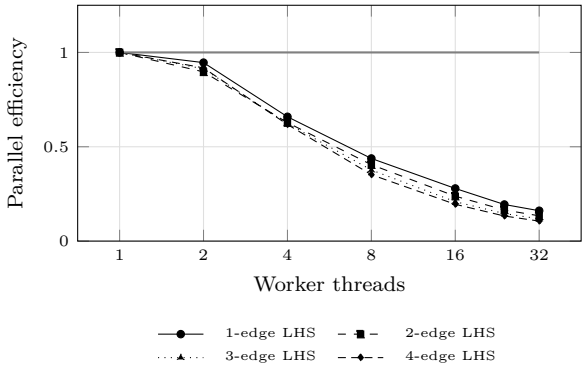


Figure 6: Wall time against worker count for the sweep's ten timed workloads, each at the depth its legend entry states. Both axes are logarithmic; the legend lists the curves in descending order of single-threaded time.

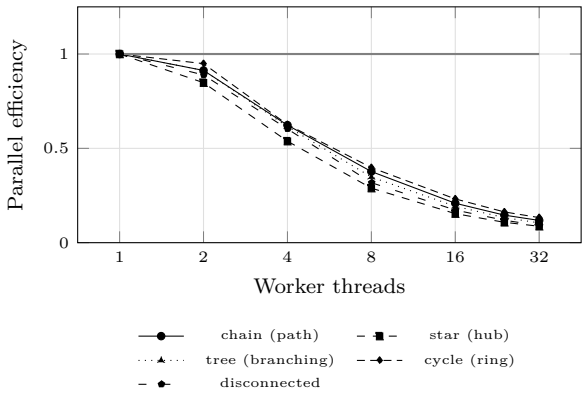
The four chain sizes run at depth seven, so the size comparison is controlled. Single-threaded wall time falls as the left-hand side grows: 93.9, 70.5, 38.5 and 20.0 s for one to four edges, following the state count at that depth (18,636,561 down to 1,831,446; Table 8). Efficiency at 32 threads falls in the same order: 0.161, 0.134, 0.118, 0.106. Table 6 measures two different rules at different depths, so its columns are not comparable with these.

At 32 threads, efficiency along the connectivity axis ranges from 0.087 to 0.133: ring 0.133, path 0.118, branching tree 0.105, disconnected 0.091 and hub 0.087. The disconnected shape also suffers from cache traffic between L3 domains. At a fixed two threads, the disconnected workload takes 976 ms when both threads share an L3 domain and 1494 ms when they do not, against 1322 ms on one thread.

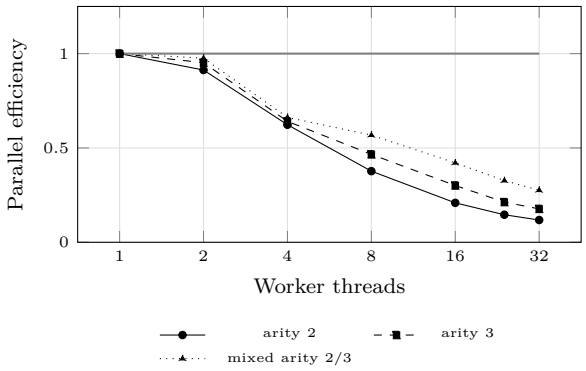
Arity gives the widest spread. With a three-edge left-hand side, efficiency at 32 threads is 0.118 at arity two, 0.177 at arity three, and 0.276, the highest in the sweep, with arities alternating between two and three (at depths seven, eight and thirteen). In a path of arity three, consecutive edges share two vertices, which fixes a match's orientation, so each state has a narrower frontier and more of its children are duplicates.



(a) Left-hand-side size: one to four edges, all binary chains.



(b) Connectivity at comparable size: path, hub, branching tree, ring, two disjoint components.



(c) Edge arity at a fixed three-edge left-hand side.

Figure 7: Parallel efficiency along the three properties of a left-hand side. Each curve is normalized by its own single-threaded time on the same binary, workload and canonicalization mode; the grey line $y = 1$ is ideal scaling.

8.7 Memory Behavior under Thread Scaling

Falling efficiency at high thread counts could come from contention on the shared structures or from the cost of acquiring memory. Table 7 gives `getrusage` figures for a workload of 4.6 seconds of single-threaded work. Across a 32× increase in workers, user time grows 3.3× and system time 7×, reaching 2.2 s of 16.6 s of CPU time at 32 workers, so the extra time is in user code. Peak resident set grows by an eighth and minor faults eighteen-fold, and system time, which follows the faults as each worker draws its own arena blocks, stays small, so memory acquisition is not the cause. The instruction count is flat across the sweep (below), so spinning is not the cause either.

Hardware counters on the EPYC host identify the cause there. On the Wolfram rule at depth seven, executed instructions are 14.5G at one thread and 15.8G at 32, while cycles double, so instructions per cycle fall from 1.44 to 0.69. The counter that grows is demand fills served from another L3 domain's cache (AMD `ls_dmnd_fills_from_sys.near_cache`): 47k at one thread and 10.6M at 32. Fills from DRAM and from the local cache hierarchy stay flat. Two experiments rule out the concurrent structures as the source. Limiting the shared table's probe length from 32 to 4 leaves the fill count unchanged and makes the run slower. A thread-private filter that removes 80% of the repeated inserts into the shared set lowers the set's share of the fills from 36% to 12% without changing the total or the wall time; the fills move to neighboring code. The traffic comes from the evolution itself: about twelve cross-domain fills per object created, because a state created on one core complex is read by workers on others, and this part has eight L3 domains.

The cost appears in wall time at the worker that first uses a second L3 domain. On the EPYC part an L3 domain holds two cores. With one worker per physical core, wall time rises from two workers to three on every large workload (`wpp` by 12.7%, `multirule` by 36.1%), and placing the third worker on an SMT sibling in the first domain removes the rise. The engine therefore fills each domain's two cores and their SMT siblings before using the next domain. This placement is faster at every worker count measured, including sixteen (`wpp` 197 to 182 ms, `multirule` 34.7 to 25.7 ms), although at sixteen it uses half as many physical cores, and it was used for every table in this section.

Table 7: User and system time, peak resident set and minor page faults against thread count, from `getrusage`.

Threads	Wall s	User s	System s	Peak RSS MB	Minor faults
1	4.57	4.40	0.30	4791	15 249
8	1.15	6.79	0.55	4869	81 326
16	0.92	9.08	0.96	5102	158 558
24	0.94	11.48	1.61	5250	197 400
32	0.97	14.45	2.15	5430	277 060

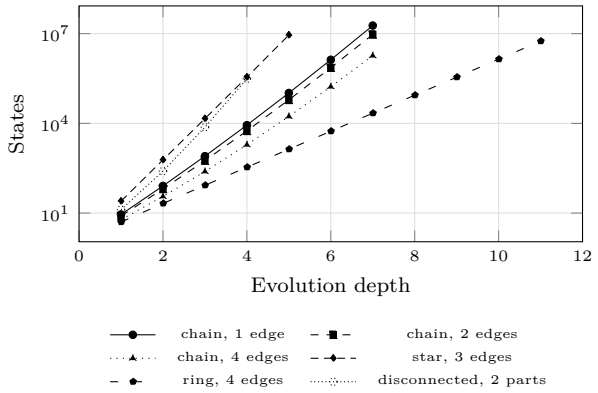
8.8 Evolution Depth and Relation Sizes

Each shape is evolved one depth at a time until a single run exceeds the collection budget, so the right-hand end of each curve in Figure 8 is the deepest depth that shape reached within the budget. Runs that filled a container are excluded, because such a run returns a truncated graph: one shape returns the same 2,097,149 states when asked for depth 9, 10 or 11. Of the 192 collected points, 41 are excluded for this reason.

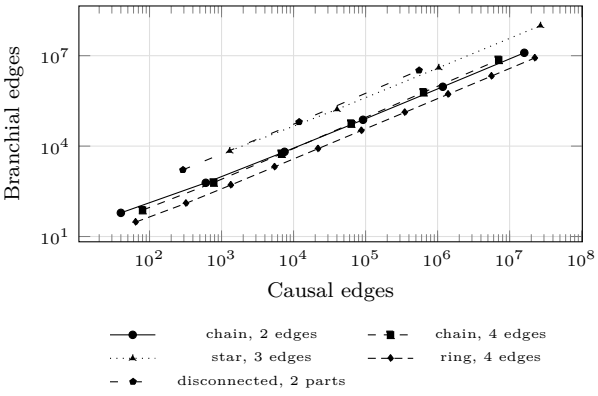
States and relation sizes vary independently. At around 300k states the two-component shape has 3.2M branchial edges while a mixed-arity chain of similar size has none,

and across the sweep the branchial count ranges from zero to 58.8M. Branchial edges connect pairs of events from the same state that share a consumed edge, so their number depends on how many matches overlap rather than on how many states exist, and a shape's position on the right panel of Figure 8 is a property of its rule. Table 8 gives the deepest point each shape reached within the budget without filling a container, so depths differ between rows. `chain2a3` and `chain2a4` have identical counts because in a path of arity three or more consecutive edges already share enough vertices to fix a match's orientation, so the two searches are the same.

A one-edge left-hand side produces no branchial edges at any depth, because two matches of a single-edge pattern cannot share an edge. Every shape with two or more edges produces them.



(a) States against evolution depth, logarithmic in states.



(b) Branchial edges against causal edges, both logarithmic; each point is one (shape, depth) run.

Figure 8: The state space and its relations, by left-hand-side shape.

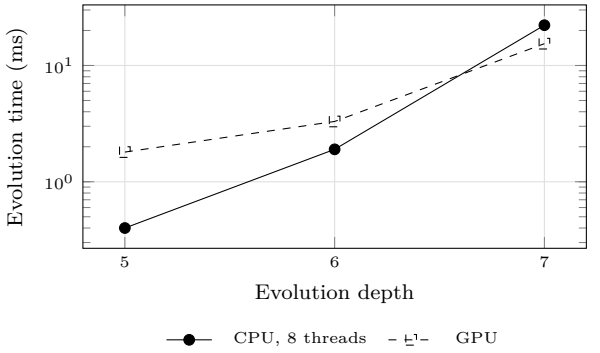
8.9 Device Comparison

Both engines run the same rule, initial state, canonicalization mode and exploration strategy, and report medians from one session (Figure 9, Table 9). At depth five the CPU is faster, because there is too little work to cover the device's fixed cost per evolution. At depth six the device is faster than one CPU thread and slower than eight. At depth 7 it is faster than both, 1.4× against eight CPU threads and more against one.

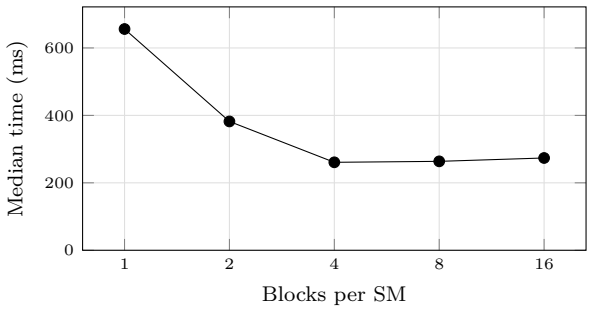
Table 11 compares the resident kernel with a per-step `evolve()` on the device, on the same workload. The resident kernel is 24.4× faster at depth four, 17.9× at five, 10.4× at six, 6.7× at seven and 20.3× at eight (603,857 states). At depths four to six the difference is a fixed cost per call:

Table 8: The deepest point reached by each left-hand-side shape: `Shape` the connectivity, `Arity` the vertices per edge, `LHS` the pattern's edge count; the remaining columns are the sizes of the three objects the run built.

Rule	Shape	Arity	LHS	Depth	States	Causal	Branchial
`chain1a2`	chain	2	1	7	18,636,561	12,825,936	0
`growth`	chain	2	1	10	4,037,914	4,037,912	0
`chain2a2`	chain	2	2	7	9,374,968	15,998,404	12,470,527
`pair`	chain	2	2	9	21,977,356	42,951,548	19,958,399
`chain2a3`	chain	3	2	8	6,725,601	11,607,482	5,764,800
`chain2a4`	chain	4	2	8	6,725,601	11,607,482	5,764,800
`chain3a2`	chain	2	3	7	4,357,969	12,187,280	12,287,285
`triple`	chain	2	3	9	21,977,356	64,928,900	37,897,842
`chain3a3`	chain	3	3	9	12,093,235	34,835,524	18,139,851
`chain4a2`	chain	2	4	7	1,831,446	7,054,476	7,264,602
`quad`	chain	2	4	9	21,977,356	86,906,252	53,818,329
`cycle3a2`	cycle	2	3	14	7,174,453	21,523,347	7,174,452
`cycle4a2`	cycle	2	4	11	5,592,405	22,369,600	8,388,606
`disc2a2`	disc	2	2	4	309,853	547,488	3,238,014
`disc`	disc	2	2	5	17,244,253	31,997,088	214,918,014
`disc3a2`	disc	2	3	4	1,045,591	3,136,752	58,835,751
`mixed2`	mixed	2	2	12	22,369,621	42,613,480	0
`mixed3`	mixed	2	3	14	7,174,453	21,441,385	4,782,968
`mixed4`	mixed	2	4	14	7,174,453	28,632,164	4,782,968
`star2a2`	star	2	2	6	6,541,573	12,579,840	24,840,738
`star3a2`	star	2	3	5	8,967,193	26,779,248	99,436,884
`star4a2`	star	2	4	5	8,433,241	33,732,864	96,982,260
`tree3a2`	tree	2	3	7	5,049,431	13,969,772	14,356,591
`tree4a2`	tree	2	4	7	11,583,495	44,294,548	87,189,943



(a) Evolution time against depth, the device against eight CPU threads (Table 9).



(b) Median time against the resident kernel's grid size (Table 12).

Figure 9: Device comparison. The device is slower than eight CPU threads at depths five and six and faster at depth seven; time stops improving beyond four blocks per streaming multiprocessor.

`evolve()` takes 32–33 ms while the state count grows from 53 to 3,867. Depth nine needs more device memory than the card has.

Table 9: End-to-end evolution time by depth on the Wolfram rule: GPU against CPU at one and at eight threads.

Steps	Raw states	Events	CPU 1t ms	CPU 8t ms	GPU ms	vs. CPU 1t	vs. CPU 8t
5	401	400	0.9	0.4	1.8	0.51	0.24
6	3867	3866	9.5	1.9	3.3	2.90	0.57
7	45317	45316	132.6	22.2	15.4	8.62	1.44

Table 10: GPU against CPU across the corpus, one row per workload at its standard depth: the single rules, the rule combinations, and the large symmetric states. Both engines run the same rules, initial states, canonicalization mode and exploration strategy; medians share one session and one iteration count.

Workload	Depth	Raw states	CPU 1t ms	CPU 16t ms	GPU ms	vs. CPU 1t	vs. CPU 16t
wpp	7	45317	132.9	15.1	15.2	8.76	0.99
binary	6	22	0.1	0.2	1.1	0.11	0.20
wolfram24	6	9820	32.5	4.1	5.1	6.33	0.81
triangle	6	4	0.1	0.2	0.9	0.07	0.21
arity3	6	7	0.1	0.2	1.7	0.05	0.11
multirule	6	49	7.2	1.8	12.0	0.61	0.15
multirule_grow	6	22	0.2	0.2	1.4	0.12	0.18
multirule_binary	6	28	0.3	0.4	1.4	0.22	0.29
cycle4	6	40	4.1	1.1	6.3	0.64	0.17
multiroot	6	104	1.6	0.6	2.7	0.58	0.24
disc2x2	6	1409	37.9	6.7	8.5	4.46	0.78
growshrink3	5	8215	20.4	2.7	3.9	5.26	0.70
wolftri	6	10037	36.5	4.2	5.4	6.72	0.78
arimix	6	265	0.8	0.3	2.0	0.39	0.16
allfour	5	25571	72.4	9.0	10.7	6.79	0.84
bigpath	2	512	15.1	2.6	21.0	0.72	0.12
bigcycle	2	514	28.0	3.8	38.4	0.73	0.10
bigstar	2	34	0.8	0.5	7.8	0.10	0.07

Table 11: The resident kernel against a per-step `evolve()`, both on the device, by depth on the same rule and initial state.

Depth	States	`evolve()` ms	Persistent ms	Speedup
4	53	32.0	1.3	24.38×
5	401	31.7	1.8	17.90×
6	3,867	33.4	3.2	10.41×
7	45,317	102.3	15.3	6.68×
8	603,857	5374.0	264.7	20.30×

8.10 Device Saturation

The resident kernel is launched with a fixed number of blocks per streaming multiprocessor, so that number is a tuning parameter. On the Wolfram rule at depth eight, wall time falls as blocks are added until four per SM; eight per SM, the default, is 1.1% slower than four, and sixteen 4.9% slower (Table 12). The last column, the driver's peak SM-busy reading, reports whether the multiprocessors were busy, not how many warps were resident; it reads 100% at every point.

Table 12: Wall time and peak SM-busy against blocks per streaming multiprocessor.

Blocks per SM	Blocks	Median ms	vs. 1 per SM
1	128	656.12	1.00×
2	256	382.06	1.72×
4	512	260.86	2.52×
8	1024	263.71	2.49×
16	2048	273.67	2.40×

8.11 Occupancy and Lane Efficiency

Table 13 gives occupancy, throughput and active lanes per warp for the resident kernel on every corpus workload, from hardware counters under Nsight Compute.

Occupancy. `cuobjdump -res-usage` on the linked binary reports that `k_persistent_evolve` uses 255 registers per thread, the architectural maximum. At 32 threads per block a block reserves 8,160 registers, and an sm_89 multiprocessor has 65,536, so eight blocks fit: 256 resident threads against a maximum of 1,536, an occupancy of 16.7%. The launch grid is also eight blocks per multiprocessor, so the two limits coincide. The measured occupancy is 14.1% to 17.3% on every corpus workload, over kernel durations that differ by a factor of seven hundred. Limiting the kernel to 128 registers halves the register allocation and leaves occupancy unchanged, so the grid, not the register count, sets it. Doubling the grid under that limit doubles residency and makes the kernel slower, increasingly so as the grid grows, because the added warps compete for the shared work queues without adding independent work.

Active lanes. The canonicalization phase runs on the whole warp: refinement's loops are spread across the lanes, and the search's serial steps (individualization, orbit closure, backtracking) run on one lane while the others wait at warp barriers. Publication, signature, quotient and deduplication steps run on one lane. On `wpp` at depth six the kernel averages 3.5 active lanes per warp out of 32. Per-block `clock64` timing shows that, on the workloads with enough work to measure, canonicalization takes most block cycles, rewriting a few percent and matching under two.

Throughput. No hardware unit is near its limit. On `wpp` at depth six SM throughput is 9.7% of peak, memory throughput 24.3% and DRAM 11.6%; across the corpus the maxima are 12.6%, 33.8% and 16.1%. Atomic operations, which the work queues use, are served by L2, whose throughput is included in the memory figure.

On the large workloads the device is therefore limited by the serial steps of the IR search, in which each step depends on the one before, and not by bandwidth, queue contention or occupancy. A separate measurement records no time spent waiting for unpublished work.

Workloads with few classes. The workloads with few states behave differently. With causal recording on, `cycle4` and `multirule` reconstruct causal relations far larger than their state counts and still leave the device idle most of the time, with SM throughput of 2.2% and DRAM 0.01% on `cycle4`, and 9.8 and 10.4 active lanes. The kernel takes four times longer on `multirule`'s 49 states than on `wolfram24`'s 9,820. At depth six `cycle4` replays 68,185 instances over 40 classes and is slower than the host, while `wolfram24` replays 86,086 instances over 9,820 classes and is faster. The instance counts are similar and the class counts differ by a factor of 250. The replay walks each class's instances on one worker, so a run with forty classes has forty workers' worth of parallel work however many instances it holds. `cycle4` starts from an initial state with many automorphisms, which collapses its states into few classes.

Causal recording. The measurements above run with causal recording off. With it on, the device and host reconstruct the same causal relation on all eight corpus workloads the device supports, at depth six: the same number of distinct pairs and the same number after transitive reduction. On the densest available workload, a three-edge disconnected left-hand side at depth three, both give 1,661,760 causal pairs and 970,560 after reduction. The two engines share only the replay core, with separate schedulers, memory layouts and queues. Both store the relation unreduced and reduce it when it is read, using one shared reduction; a finite DAG has one transitive reduction, so the result does not depend on the order in which pairs arrive. The reduction is computed once per source event, since every edge leaving an event shares the set reachable from it in two or more steps, and when event identifiers increase along every causal edge the search stops at the source's largest target. Branchial pairs are not stored: both engines derive them from the applications when they are read.

Host traffic. Across every evolution traced, including twenty-four consecutive evolutions with the reconstruction on, no copy, allocation or memset runs while the resident kernel runs.

When raw events are not requested, the replay does not run (Section 6.1). With the replay on, `multirule` and `cycle4` do not complete depth seven; without it both complete, with the same state and event counts.

Table 13: **Resident-kernel occupancy, throughput and active lanes across the corpus**, from hardware counters under NVIDIA Nsight Compute: means over the profiled launches of the resident kernel. Durations are the profiler's, which serializes execution and fixes clocks, and are not comparable with the wall times of Table 10.

Workload	Depth	Launches	Duration (µs)	SM %	Mem. %	DRAM %	Lanes/warp	Occ. %
`allfour`	6	6	187381.7	8.46	21.51	9.75	3.00	17.03
`disc2x2`	6	7	178994.3	3.88	3.95	0.17	6.05	16.84
`multirule`	6	5	113454.0	2.24	2.30	0.02	10.41	17.25
`bigcycle`	2	9	86642.2	7.93	9.11	0.15	2.83	16.65
`bigpath`	2	11	82264.5	6.71	9.14	0.17	3.16	16.66
`growshrink3`	6	12	81015.0	10.86	33.27	16.08	3.14	16.99
`cycle4`	6	3	73393.3	2.23	2.72	0.01	9.84	16.39
`wolftri`	6	3	17210.0	7.46	20.83	8.46	2.74	14.12
`multiroot`	6	3	13420.0	2.38	2.43	0.04	8.87	15.54
`wolfram24`	6	3	9086.7	12.59	33.75	12.88	2.71	16.41
`bigstar`	2	3	8806.7	2.44	2.43	0.01	9.79	16.73
`wpp`	6	3	3900.0	9.65	24.25	11.57	3.45	16.71
`arimix`	6	3	3213.3	2.99	4.29	0.21	8.08	16.58
`binary`	6	3	2676.7	2.20	2.22	0.03	11.17	16.77
`arity3`	6	3	2106.7	2.20	2.17	0.03	11.59	16.42
`triangle`	6	3	263.8	2.60	2.95	0.21	11.59	16.28

8.12 Quotient Exploration

Quotient exploration expands each canonical state once, at its shallowest depth, so it produces far fewer events than full expansion, and both give the same canonical state set (Table 14). At depth seven quotient exploration produces 28 events where full expansion produces 5,913, and the reconstruction recovers all 5,913, with their causal edges and branchial pairs, from those 28. The reconstruction's cost is below the resolution of this measurement: at most depths the `quot.+recon` time is at or below the `quot.` time, which is run-to-run variation on sub-millisecond timings.

Table 14: Events and wall time, quotient exploration against full expansion, by depth on the Wolfram rule; at every depth the reconstruction gives full expansion's event and causal counts exactly.

	Full capture			Quotient		Quotient + reconstruction	
Depth	Events	Causal	ms	Events	ms	Events	ms
2	3	4	0.3	3	0.1	3	0.1
3	9	16	0.1	6	0.1	9	0.1
4	33	64	0.3	10	0.1	33	0.1
5	153	304	0.5	15	0.1	153	0.1
6	873	1744	1.0	21	0.2	873	0.2
7	5913	11824	3.8	28	0.3	5913	0.3

8.13 Ablation

Match forwarding is the one run-time switch that changes speed without changing the answer. The state and event identity modes change which answer is computed, so a time ratio between them compares different computations; Section 3.4 reports how their results differ, and Table 14 reports quotient exploration. The static analysis of Section 5.4 turns forwarding on or off per rule set. Table 15 gives best-of-N wall times for both settings. On the one-edge left-hand side (depth eight) the analysis turns forwarding off, and forwarding on is 1.04× slower. On the two-edge left-hand side (depth six) it turns forwarding on, and forwarding on is 1.01× slower than off. Both settings give the same state and event counts on both workloads.

Table 15: Match forwarding on and off: one-edge left-hand side at depth eight, two-edge at depth six. Best-of-N wall times; both settings give identical state and event counts.

Contribution	Arm A	A ms	Arm B	B ms	Ratio
Match forwarding, one-edge LHS	on	14.7	decided off	14.2	1.04
Match forwarding, two-edge LHS	off	12.4	decided on	12.5	0.99

8.14 Event Identity

Table 16 evolves each workload once, single-threaded in the exact state mode, and counts distinct events under each of the three conventions of Section 3.4: by endpoint states, by consumed and produced edges, and by distinct applications. Each convention refines the one before, so the counts do not decrease from left to right, and they differ on most workloads: on `binary-growth` they are 5, 8 and 9. Every other event count in this paper counts distinct applications.

Table 16: Distinct events per identity convention on each corpus workload.

Workload	Steps	Canon. states	Endpoint states	Consumed/produced edges	Distinct applications
binary-growth	3	6	5	8	9
wolfram-2to4	2	4	3	4	6
triangle	2	3	2	4	4
reductive-2to1	3	4	3	6	15
idempotent-2to2	3	1	1	3	3
self-loop	3	2	1	1	1
mixed-arity	3	4	3	3	3
arity3-growth	3	9	9	9	9
disconnected-lhs	3	1	1	10	14
arity4-growth	3	9	9	9	9
mixed-arity-lhs	3	2	1	1	1
mixed-arity-init	3	4	4	4	4
arity3-to-2	3	4	4	4	4
arity1-with-binary	3	2	1	1	1
cycle4-automorphic	3	4	3	15	144
star4-automorphic	2	5	4	12	24
multi-rule	2	10	11	11	11

8.15 Instruction Profile

Executed instruction counts are deterministic for a single-threaded run and unaffected by other load, so they attribute time to parts of the engine. Table 17 comes from one `callgrind` run of a single-threaded depth-six evolution of the Wolfram rule under quotient exploration with the raw relations reconstructed, the CPU benchmark's default configuration. Canonicalization (31.1%) and the quotient reconstruction (19.3%) are the two largest parts; matching, its join and the rewrite together are 10.6%, libc's copies, fills and heap calls 9.4%, and the concurrent sets and maps 4.0%. The unattributed 6.5% is functions below the reporting threshold and code not assigned to a group.

Table 17: Executed instructions by component: a single-threaded depth-six evolution of the Wolfram rule under quotient exploration with reconstruction, under `callgrind`, grouped by the algorithm each function belongs to. The per-function profile covers 99.0% of the program total; the rest is below the annotator's reporting threshold. The benchmark-harness row is the corpus set-up the benchmark performs before the evolution.

Component	Instructions	Share
Canonicalization (IR and orbits)	45,513,296	31.1%
Quotient reconstruction (replay, causal DP, TR)	28,307,967	19.3%
Match expansion, join and rewrite	15,475,736	10.6%
Hypergraph storage	15,385,800	10.5%
libc copies, fills and heap	13,764,099	9.4%
Arena allocation	7,388,170	5.0%
Concurrent sets and maps	5,813,440	4.0%
Benchmark harness	3,443,355	2.4%
Job system and work stealing	432,725	0.3%
Unattributed	9,460,454	6.5%
Total	146,444,166	99.0%

8.16 Limitations

Two machines. The left-hand-side sweep, the occupancy table and the hardware-counter measurements ran on the AMD EPYC host and everything else on the Intel i9-14900K desktop of Section 8. The scaling results depend on each host's memory system and core count, and the thread count at which efficiency declines differs on other hardware.

Quotient reconstruction grows with the raw system. Quotient exploration visits one state per isomorphism class, but the reconstruction of raw observables creates as

many instances as the raw system has. On a two-rule set whose canonical state count grows linearly with depth (three to eight states at depths two to seven), the reconstruction took 0.17, 0.22, 0.62, 7.36, 144.78 and 4981.05 milliseconds, about twentyfold per step. Runs that request only canonical states skip it.

Two comparison points, one of them our own. The engine is compared with the `Wolfram/Multicomputation` paclet's `MultiwaySystem` and with this project's reference implementation. `MultiwaySystem` is the only other implementation we know of that computes the deduplicated states graph under isomorphism; SetReplace computes a different object (Section 2). The reference implementation was written by the same author as the engine, from the same definition.

Workload scope. The oracle corpus has 17 workloads (Tables 2, 3 and 16), and the left-hand-side sweep of Section 8.6 has 24 shapes (Table 8). Both were built to cover rule shapes (productive, reductive, idempotent, mixed arity, disconnected and highly symmetric), not sampled from rules used in practice, and the observed C/R and parallel efficiency describe those shapes.

Scope of the concurrency results. The model-checked properties hold for the stated thread and operation counts, which are small, and the determinism gate covers the configurations it runs.

Worst-case cost. IR is exponential in the worst case and runs on every state in the exact mode. States with large automorphism groups are where that cost occurs, and Proposition 3.3 bounds how much a cheaper pre-filter can save.

9 Verification of the Concurrent Code

The engine holds no lock, so its correctness under concurrency rests on memory-ordering arguments. A stress test samples only a small fraction of the possible interleavings: a determinism gate of 150 runs explores almost none of the schedules its thread count admits. The concurrent code is therefore checked by two model checkers in addition to the test suite.

Model checking under weak memory. GenMC [23] enumerates every execution of a bounded concurrent program under the RC11 memory model [24]: for the given thread count, operation count and inputs, it explores every interleaving and every choice of which write each read observes. A clean result therefore covers all executions within the bound. Three harnesses are weaker. Two bound the number of context switches, which the checker supports only under sequential consistency, so they cover the sequentially consistent executions. The third samples executions because its shape is too large to enumerate, and the same property is checked exhaustively at a smaller shape by another harness.

Each harness is a small program that includes the engine's headers and calls its functions, so it checks the engine's code rather than a separate model, and it stops compiling when the code it checks changes shape. Two harnesses check protocols inside functions the checker cannot load, and contain verbatim copies of that code running over the engine's structures.

The harnesses check the following properties, each of which would fail without an error:

- *Concurrent map:* get-or-create has exactly one winner and returns the winner's value to every caller, including when a resize runs concurrently with the inserts that caused it and across two successive resizes; the "inserted" flag comes from the publishing exchange, so repeated offers of one value are not each reported as insertions.
- *Key set:* two threads inserting one key while a third forces a resize produce exactly one reported insertion; an enumeration during a resize reports each key once; and a membership query for a key inserted before the query began answers present, including while the key is being copied to the new table. This last property requires walking the chain of tables from the oldest; the harness contains a twenty-execution counterexample for newest-first order.
- *Work-stealing deque:* the owner and a thief never take the same item, checked where the two ends address the same slot.
- *Match rendezvous:* a match is claimed exactly once and never dropped on a collision, and every pair of an instance and a match recorded for its class is replayed.
- *Per-instance application list:* of two applications linked into one list, exactly one sees the other, so each branchial pair is emitted once (Section 6.1).
- *Arena:* no two workers receive the same arena index, and the shared allocation path never hands out memory a worker's private cursor owns.
- *Scheduler:* a worker that waits is always woken. Removing either of the two sequentially consistent fences in the submit and wait protocol makes the checker report a non-terminating spin loop.

Six harnesses check the GPU kernel's protocols under the scoped extension of the memory model: the ring buffer's exactly-once hand-off, termination detection without early exit, the ring and the termination detector together, the hash table electing one inserter, the replay rendezvous meeting every pair, and the causal dynamic program's producer–transition rendezvous.

Harnesses that link the whole engine construct an engine and add a rule; at the stated bound the checker reaches the end of each (442 complete executions), so the composed construction is covered exhaustively.

To run the engine through GenMC, the checker was modified; the patch against release 0.17.0 is shipped with the engine. It has four correctness fixes (`operator new[]` treated as an allocation; typed promotion of `memcpy` and `memset`; the failure ordering for a failed compare-exchange's read; and no liveness report while a thread is blocked by an assume, upstream pull request 58), a second pass of memory-intrinsic promotion after scalar replacement, and instrumentation: a progress report, and an attribution of the state-space estimate to the source lines whose reads and writes produce it.

TLA$^+$ models. Four protocols are also specified in TLA+ and checked with its model checker [25]. Every protocol step in progress is held in a set from which any enabled step may fire, so the checked interleavings cover every worker count; the bound is on the number of states instead. Each specification is a transcription of the code and names the code it transcribes. Each has a broken configuration that the checker must reject, which shows that the specification can detect the failure it is about.

Match forwarding is the property whose failure costs the most: a lost match removes the whole subtree below it and

leaves the run internally consistent. The specification is checked in four configurations (Table 18), which differ in whether the process that wins a match's claim both stores and propagates it, and in whether submission is batched. The shipped configuration is complete at the bound. Without ownership and without batching the property fails on a chain $s_0 \to s_1 \to s_2$: a match is found at the root after s_2's pull has run; s_1's pull wins the claim and stores the match without propagating it; the root's push finds the claim taken and does not recurse, so s_2 never receives the match. Batching alone hides this failure at the bound, so ownership is required independently of batching.

Table 18: **Forwarding completeness under TLA+**, exhaustive at the bound of four states, three matches and three edges, with a discovery mid-chain that enables a grandchild. *Ownership*: the claim winner both stores and propagates the match; *batched*: a state's matches are submitted together after its matching completes.

Configuration	Ownership	Batched	Verdict	Distinct states
shipped	yes	yes	complete	117,005
calibration	no	yes	complete	85,777
calibration	yes	no	complete	479,005
calibration	no	no	violated (as required)	trace depth 13

Depth relaxation (Section 5.6): at quiescence the set of expanded states is exactly the set whose shortest-path depth is below the budget, including when a steered step lowers the budget for one pass. The broken configurations derive a child's depth from its parent's depth at claim time, and claim states against a bound different from the one the match task is admitted under; both violate the property.

Termination (Section 5.6): the job system never reports completion while work remains, provided a job submits all of its work before it is counted as complete. The broken configuration, in which a job submits work after it has been counted, violates the property.

Segmented array: the array that stores states, events, edges and application lists grows by segments, and if a segment exists then every segment below it exists. The shipped protocol is clean at 2,284 distinct states; allocating only the requested segment violates the property. Three appenders are needed to expose the failure: with two, each claims once, and the index that would reveal the gap cannot be reached before the lower segment is published. GenMC cannot construct this structure, so it is checked in TLA+.

Sanitizers. The test suite runs under ThreadSanitizer, AddressSanitizer and UndefinedBehaviorSanitizer, and the device suites under the CUDA memory and initialization checkers; all five report nothing and fail the build on a finding. The initialization checker catches host reads of device memory that no kernel wrote, which return whatever the allocator last left there, so a test on such a value passes or fails depending on allocation history.

10 Conclusion and Future Work

We have presented a multiway hypergraph rewriting engine for CPU and GPU built on exact canonicalization and lock-free concurrency. On the same rule and initial state as the `Wolfram/Multicomputation` paclet's `MultiwaySystem`, and with identical state and causal-edge counts, its C++ core is 1,696× faster at depth 4 and 50,384× faster at depth 7 (Table 4); through the paclet's Wolfram interface it is still 3081× faster. On the CPU, evolution reaches a speedup of 10.2× at 32 workers (Table 5); on a part with eight L3 domains it is limited by cache traffic between them rather than by contention on the shared structures (Section 8.7). On the GPU, the resident kernel is 1.4× faster than eight CPU threads at depth 7 (Table 9), and is limited by the serial steps of the canonicalization search.

10.1 Future Directions

Distributed computation: The engine runs on one machine; distributing it would allow state spaces larger than one machine's memory.

Rule search: Searching for rules whose evolutions have given properties, such as a dimension or causal invariance, is an open problem.

Verification beyond the bound: The model-checking results hold for bounded programs under RC11, and for four protocols at any worker count in transcribed TLA+ models. A mechanized proof over the implementation would cover unbounded thread counts.

10.2 Availability

The engine is available as a C++/CUDA library and as a Wolfram Language paclet. It can also emit its evolution as a stream of events (states, rewrites, causal and branchial edges) as they are produced, for applications that render or process the evolution while it runs.

- Source code: https://github.com/WolframInstitute/HypergraphRewritingEngine
- Wolfram Language paclet: `WolframInstitute/HypergraphRewriteEngine`, installable from the release page of the repository

Acknowledgments

The author thanks Stephen Wolfram for foundational work on the Wolfram Physics Project. This work was supported by the Wolfram Institute.

References

[1] Stephen Wolfram. A project to find the fundamental theory of physics. https://www.wolframphysics.org, 2020. Wolfram Media.

[2] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014.

[3] Zhengyi Yang, Wenjie Zhang, Xuemin Lin, Ying Zhang, and Shunyang Li. HGMatch: A match-by-hyperedge approach for subgraph matching on hypergraphs. In *IEEE International Conference on Data Engineering (ICDE)*, pages 2063–2076, 2023.

[4] Hartmut Ehrig, Karsten Ehrig, Ulrike Prange, and Gabriele Taentzer. *Fundamentals of Algebraic Graph Transformation*. Monographs in Theoretical Computer Science. Springer, 2006.

[5] Grzegorz Rozenberg, editor. *Handbook of Graph Grammars and Computing by Graph Transformation*, volume 1. World Scientific, 1997.

[6] Detlef Plump. The graph programming language GP. In *Algebraic Informatics*, pages 99–122. Springer, 2009.

[7] Johannes Köbler, Uwe Schöning, and Jacobo Torán. *The Graph Isomorphism Problem: Its Structural Complexity*. Birkhäuser, 2012.

[8] László Babai. Graph isomorphism in quasipolynomial time. *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing*, pages 684–697, 2016.

[9] Boris Weisfeiler and Andrei Lehman. The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, 2(9):12–16, 1968.

[10] Nino Shervashidze, Pascal Schweitzer, Erik Jan Van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research*, 12:2539–2561, 2011.

[11] Wolfram Physics Project. SetReplace: A Wolfram language package and C++ library for hypergraph rewriting. https://github.com/maxitg/SetReplace, 2020.

[12] Wolfram Research. Wolfram/Multicomputation. Wolfram Language Paclet, 2023. URL https://resources.wolframcloud.com/PacletRepository/resources/Wolfram/Multicomputation/. Provides `MultiwaySystem`.

[13] Ha-Nguyen Tran, Jung-jae Kim, and Bingsheng He. Fast subgraph matching on large graphs using graphics processors. In *Database Systems for Advanced Applications*, pages 299–315. Springer, 2015.

[14] Xuhao Chen, Roshan Dathathri, Gurbinder Gill, and Keshav Pingali. Pangolin: An efficient and flexible graph mining system on CPU and GPU. *Proceedings of the VLDB Endowment*, 13(8):1190–1205, 2020.

[15] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. Gunrock: A high-performance graph processing library on the GPU. *ACM SIGPLAN Notices*, 51(8):1–12, 2016.

[16] Xuhao Chen and Arvind. Efficient and scalable graph pattern mining on GPUs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[17] Gramoz Goranci, Adam Karczmarz, Ali Momeni, and Nikos Parotsidis. Fully dynamic algorithms for transitive reduction. In *International Colloquium on Automata, Languages and Programming (ICALP)*, volume 334 of *LIPIcs*, pages 92:1–92:20, 2025.

[18] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.

[19] Jaco van de Pol and Michael Weber. The fastest term rewrite engine. *Electronic Notes in Theoretical Computer Science*, 343:169–187, 2019.

[20] Gian-Carlo Rota. The number of partitions of a set. *The American Mathematical Monthly*, 71(5):498–504, 1964.

[21] Albert Atserias, Martin Grohe, and Dániel Marx. Size bounds and query plans for relational joins. *SIAM Journal on Computing*, 42(4):1737–1767, 2013.

[22] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. Worst-case optimal join algorithms. *Journal of the ACM*, 65(3):16:1–16:40, 2018.

[23] Michalis Kokologiannakis and Viktor Vafeiadis. GenMC: A model checker for weak memory models. In *Computer Aided Verification (CAV)*, volume 12759 of *Lecture Notes in Computer Science*, pages 427–440. Springer, 2021.

[24] Ori Lahav, Viktor Vafeiadis, Jeehoon Kang, Chung-Kil Hur, and Derek Dreyer. Repairing sequential consistency in C/C++11. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 618–632. ACM, 2017.

[25] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[26] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2012.

[27] Maged M. Michael. High performance dynamic lock-free hash tables and list-based sets. In *Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 73–82, 2002.

[28] Timothy L. Harris. A pragmatic implementation of non-blocking linked-lists. In *International Symposium on Distributed Computing (DISC)*, pages 300–314. Springer, 2001.

[29] David Chase and Yossi Lev. Dynamic circular work-stealing deque. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 21–28, 2005.

[30] Nhat Minh Lê, Antoniu Pop, Albert Cohen, and Francesco Zappa Nardelli. Correct and efficient work-stealing for weak memory models. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 69–80. ACM, 2013.

A Complexity Analysis Details

This appendix gives the cost bounds for the two operations that dominate a run: canonicalizing a produced state, and finding the matches of a rule in a state. In both, the worst case depends on the input: on the state's symmetry for the first, and on the left-hand side's shape for the second. Table 19 gives the per-operation costs of the concurrent structures.

A.1 Canonicalization Complexity

Individualization–refinement interleaves color refinement with the individualization of a vertex in a non-singleton cell, and searches the resulting tree, pruned by the automorphisms it finds. It is polynomial on states that refinement alone resolves and, like nauty and Traces, exponential in the worst case, which occurs on states with large automorphism groups. The exact mode runs it on every state, and it accounts for 31.1% of executed instructions together with the orbit computation (Table 17). By Proposition 3.3, a cheaper hash in front of it can skip at most a fraction C/R of the calls.

A.2 Pattern Matching Complexity

For a left-hand side of $|L|$ edges and a state of N edges, the edge-at-a-time join of Section 4.3 binds one pattern edge per step, and each step after the first draws its candidates from the incidence list of a vertex already bound. On left-hand sides of one or two edges the join meets the AGM bound; on a cyclic left-hand side of three edges it performs $\Omega(N^2)$ work against a bound of $N^{1.5}$ (Section 4.4).

B GPU Implementation Details

B.1 GPU Canonicalization

The device runs the same individualization–refinement source as the host, compiled for both, so the two compute the same canonical hash for every state. The device-specific code flattens a state into the canonicalizer's input and allocates its scratch.

A worker canonicalizes one state per call. The call's scratch is sized from the state's own edge and occurrence counts and taken from the device arena of Section 7.2, so there is no maximum state size.

The exact path has no approximate fallback. A state it cannot canonicalize, because the arena is exhausted or the search needs more individualization depth than the device allows, is reported as a capacity overflow and the host re-runs the evolution with larger capacities; it is never keyed by a coarser invariant. A coarser invariant would merge non-isomorphic states: a 1-dimensional Weisfeiler–Leman hash gives the same value to the prism graph and $K_{3,3}$, and to the 4×4 rook's graph and the Shrikhande graph. The only other device-side state key is the content-ordered key of the AUTOMATIC mode, which uses the host's rule.

A device-side event signature uses a raw edge identifier when an edge's canonical rank is not yet available, and each such substitution is counted, so a difference in event totals between devices can be traced to it.

B.2 GPU Pattern Matching

One block matches one (state, rule) pair, and its threads divide the candidates for the first pattern edge. The device indexes the edge pool twice, by signature and by vertex, and filters both by the state's edge set. The signature index gives the candidates for the first pattern edge. Each later edge takes its candidates from the vertex index, through a *pivot variable*: a left-hand-side variable already bound when that edge is matched, which the connected match order guarantees. The pivot's incidence list holds as many entries as the vertex's degree, typically a few, where a signature bucket on a dense state holds thousands. This division follows HGMatch [3]. The host answers the same two queries from the state's ancestry (Section 4.2).

Distinct pattern variables may bind to the same vertex, so a pattern signature is compatible with every coarsening of its partition. The compatible signature hashes of each pattern edge are computed once on the host when the rule is added.

B.3 Device Work Queues and Termination

The device work queues are bounded lock-free multi-producer, multi-consumer ring buffers in device memory. The same workers produce and consume (a rewrite produces states, which become match tasks), and the queues are bounded because device memory is reserved before the launch.

Each slot holds a sequence number that says whose turn the slot is. Slot i starts at i. A producer reserving position p finds the slot free when its sequence is p and publishes by setting it to $p + 1$; a consumer reserving p finds an item when the sequence is $p + 1$ and releases the slot by setting it to p plus the capacity. A sequence below the position means the queue is full for a producer or empty for a consumer. The sequence advances by one capacity per lap, so the tests stay exact across wraps. The producer's release store and the consumer's acquire load order the hand-off, so the head and tail cursors are relaxed.

A position is reserved by compare-and-swap on the cursor. An unconditional increment followed by a rollback can land on a slot another thread has taken, which loses an item or delivers it twice. A lost item stops the run from finishing, because the termination detector waits for its completion.

The resident kernel detects termination. Each queue has two monotonic counters, work pushed and work drained, incremented with release ordering and read by the detector with acquire ordering. All pushed and drained counts can be equal while a task that has just finished has not yet pushed its successors, so the detector requires equality on two readings separated by a backoff interval. Workers read the exit flag with acquire ordering at the top of their outer loop.

A wrong termination would truncate the evolution without an error, so the device result is compared with the host's on every corpus workload.

B.4 Memory Management

A resident kernel cannot call the host allocator, so device memory is reserved before the launch. Edge and state pools are pre-allocated with atomic counters, and scratch whose size depends on the state, principally the canonicalizer's arrays, comes from a device bump arena.

Each resident block holds one arena slot at a time and grows it to the largest state it canonicalizes, so arena demand grows with the grid, which defaults to eight blocks per streaming multiprocessor. The arena is one shared pool, sized for the average slot across blocks. There is no free operation: a block reuses its slot and claims a larger one only when a state needs it, so with doubling each block wastes at most one slot. The arena is reset at the end of the run.

A state that overflows a pre-sized region is reported through a device-side error channel. The host then re-runs the evolution with doubled capacities, up to eight times, and returns the last attempt's partial result with a warning if it still overflows (Section C.3).

C Data Structure Specifications

C.1 Lock-Free Data Structures

The structures below use standard non-blocking techniques [26]: compare-and-swap insertion into an open-addressed table [27] and lock-free list linkage [28].

ConcurrentMap⟨K, V⟩: An open-addressed hash table with compare-and-swap insertion. Lookup is wait-free and insertion lock-free; entries are never deleted.

ConcurrentKeySet⟨K⟩: A set. One compare-exchange from the empty sentinel to the key decides both presence and which caller inserted it. An insert first scans the chain of superseded tables for a settled copy of its key, and an insert that lands in a table that is no longer the newest is repeated in the newest, so no key is inserted twice. Growth copies each key into the new table and then seals its old slot with a second reserved key, so each key is enumerated once. A table whose slots are all sealed is skipped, since sealing removes the empty slots at which a linear probe stops. Membership queries walk the chain from the oldest table, the order the reader-visibility harness of Section 9 checks. The quotient reconstruction's membership marks use this set (Section 6.1).

SegmentedArray⟨T⟩: Dynamically growing array stored in fixed-size segments. Append-only with atomic size tracking.

LockFreeList⟨T⟩: Append-only linked list for index entries. Supports concurrent iteration via snapshot semantics.

Work-stealing deque: A bounded Chase–Lev deque [29], with the memory orderings of Lê et al. [30]. Each worker pushes and pops at its own end of its deque and, when idle, steals from the other end of another worker's. Following HGMatch [3], evolution is a dataflow over these deques: a scan task seeds a join, expansion tasks extend it, and completed matches become rewrite tasks.

Job pool: Tasks come from a per-thread slab, so submitting a task makes no call to the global allocator. Allocation and a release on the same thread use the owner's private free list, with no atomic operation. A task released on another thread, which work stealing makes common, is pushed onto the owner's foreign list by compare-and-swap, and the owner takes the whole list with one exchange when its private list is empty. This is free of the ABA problem: producers only prepend and the single consumer takes the entire list. Each slot names its pool in a header, and an object too large for a slot is allocated by the general allocator. Pools are never destroyed; when a thread exits its pool goes to a fixed array of claimable slots, because a stack of free pools would be exposed to ABA on its head.

Injector. Work submitted from outside a worker, and work that overflows a worker's deque, goes to one shared double-ended queue. Its state is one 64-bit word holding a 32-bit tag and two 16-bit indices, so one compare-exchange updates both ends and two pops racing for the last item cannot both succeed. The indices take 16 bits each for a capacity of 32,768 entries; a 128-bit compare-exchange is not lock-free on the supported targets. The tag detects an index pair that has returned to an earlier value.

The 32-bit tag wraps. A stale item is taken twice only if the tag completes a full cycle and both indices return to the values one popper read, all between that popper's load and its exchange, a few instructions. At eight threads the queue sustains 8.16×10^6 operations per second, so 2^{32} operations take about 500 seconds, which is how long the popper would have to be descheduled inside those instructions. The shared word costs throughput: 1.33×10^8 operations per second at one thread against 8.16×10^6 at eight. The per-worker deques are not shared in this way, so the injector holds only overflow and external submissions.

Neither queue blocks. If every worker blocked in a push, no worker would pop and the queues would never drain, and a full local deque and a full injector together are reachable, because expansion submits from inside running jobs. A job that finds both queues full runs on the calling thread.

A worker with no work waits on a 32-bit address through `futex` on Linux, `WaitOnAddress` on Windows or `os_sync_wait_on_address` on macOS, and is woken by the thread that submits work. The engine uses no mutex or condition variable on these platforms; one is compiled only on a platform that has none of the three primitives.

Serial mode. The scheduler can run with no worker threads. Every job then goes to the injector, and the thread that waits for completion runs the jobs in first-in-first-out order, so a serial run is deterministic. This is the single-threaded mode and the WebAssembly target, which cannot spawn threads. The engine above the scheduler is the same code in both modes.

C.2 Memory Model

A producer appends the new edges, inserts their ids into the state's edge set, issues a release fence, and then appends the state. A consumer that finds the state through the canonical map's acquire load issues an acquire fence before reading the edge set. A reader of a published state therefore sees its complete edge set.

C.3 Behavior at Configured Limits

When a run reaches a configured container capacity or a state or event limit, it returns the part of the evolution computed so far with a warning naming the limit; it does not throw. Both backends behave this way. A continuable run stopped by a state or event limit keeps the work the limit cut short, so raising the limit and continuing gives the same result as a run that never stopped. An option that a build does not implement is reported by a warning, and the result is computed without it.

Table 19: **Per-operation costs of the concurrent data structures.** Amortized where the structure grows; n is the number of elements held.

Structure	Insert	Lookup	Delete	Memory
ConcurrentMap	$\mathcal{O}(1)$ amort.	$\mathcal{O}(1)$	N/A	$\mathcal{O}(n)$
SegmentedArray	$\mathcal{O}(1)$	$\mathcal{O}(1)$	N/A	$\mathcal{O}(n)$
LockFreeList	$\mathcal{O}(1)$	$\mathcal{O}(n)$	N/A	$\mathcal{O}(n)$
SparseBitset	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n/64)$